

Capability Scaling-Down Laws of LLM Compression in Agent Systems

Anonymous authors

Paper under double-blind review

Abstract

Specialized workloads need specific capabilities, and the models that serve them are usually dense sources scaled down by pruning, quantization, or distillation; choosing a configuration today means measuring each one. We develop and evaluate empirical, capability-conditioned models of the compression response: for math, code, and multi-hop QA we predict the conditional loss after compression from the initial model state (parameter count, pretraining tokens, and the dense capability losses of the source or initial student) and the configuration of the method. Relations are built on a controlled panel of Pythia checkpoints whose size and pretraining stage vary independently, compared with alternatives on the same inputs, and validated with predictions frozen before new configurations and model states are measured. A five-parameter pruning form predicts math and code at unseen densities inside the fitted range on three new checkpoints (0.24 nats mean absolute error against 0.45 for a source-free strength curve) and fails beyond the range and for QA; with the quantization group size as a second axis a low-order form predicts unseen bit-widths and group sizes on the development states; for distillation the student state carries the math response and a reuse-count term carries QA on an unseen pool. Transfer to a source five times larger fails for QA and is on par with source-free references for math. Used as loss constraints, the laws select a compression method and setting for a held-out model and storage budget within 0.2 nats of the measured optimum for math and code. We release the relations with their coefficients, tested ranges, registered predictions, and measurements.

1 Introduction

Deployed language-model agents are specialists. A coding agent lives on code synthesis and tool use, a retrieval assistant on multi-hop question answering, and both pay for every token of inference. The model that serves such a workload is usually not a frontier model but a dense source scaled down by pruning, quantization, or distillation, and choosing the route and its strength today means running each candidate configuration through capability benchmarks.

That search cannot be replaced by a single compression ratio, because compression does not shrink capabilities uniformly. On twelve public models from five series, the same pruning density leaves QA the least-damaged capability in Qwen3 and OLMo-3-32B, where it improves, and the most-damaged one in the Gemma series; damage at the same density spans two orders of magnitude across series, and 3-bit quantization collapses twelve of fourteen models but not OLMo-3-32B (Appendix F). Predicting what survives requires both the initial model state and a per-capability endpoint.

We therefore ask how the capability-conditioned loss after compression can be predicted from the initial model state and the configuration of the method. The endpoint is the conditional loss of each capability on a fixed probe set, and every prediction is written relative to a reference measured beforehand, the dense source for pruning and quantization and the initial student for distillation (Eq. equation 2). The admitted inputs are the parameter count, the

pretraining token count where disclosed, the dense capability losses of the reference, and the intervention setting; no measurement of the compressed target enters a prediction.

The relations are developed on a controlled panel of Pythia checkpoints whose size and pretraining stage vary independently, by comparing low-dimensional functional forms that use the same inputs, and they are tested by freezing coefficients and predictions before new configurations and new model states are measured. This separates three questions that a single accuracy number blurs: whether the initial-state inputs help, whether a particular functional form adds anything over a same-input alternative, and how far the relation transfers.

The outcome is a set of method-specific empirical relations with measured ranges. For pruning, a five-parameter power form in the removed fraction with a source-conditioned amplitude predicts math and code at unseen densities inside the fitted range, ties a per-density regression, and fails beyond the range and for QA. For quantization, bit-width alone supports only a per-bit source regression, but with the quantization group size as a second axis a low-order form predicts unseen bit-widths and group sizes on the development states. For distillation, the student state carries the math response and a reuse-count term carries QA on an unseen pool, with a budget-dependent bias. Our contributions are (1) the prediction problem, its admitted inputs, and a development-then-validation design that separates input value, form value, and transfer range; (2) the per-method relations, their coefficients, and their independent tests against same-input and source-free baselines; and (3) a demonstration that the laws, used as loss constraints, select a method and setting for a held-out model and budget close to the measured optimum, with the capabilities and settings the relations cover and where they stop, and with the measurements, registered predictions, and evaluation code released.

2 Experimental Setup

2.1 Capability-Conditioned Loss

We measure three capabilities, mathematical reasoning, code generation, and multi-hop question answering, each through a fixed probe set $\mathcal{P}_c$ of prompt and reference pairs drawn once from held-out splits of MATH-500, MBPP, and 2WikiMultihopQA. The observable is the token-weighted conditional cross-entropy of the reference completion,

$$L_c(w) = \frac{1}{T_c} \sum_{(x,y) \in \mathcal{P}_c} \sum_{t=1}^{|y|} -\log p_w(y_t \mid x, y_{<t}), \quad T_c = \sum_{(x,y) \in \mathcal{P}_c} |y|, \quad (1)$$

with prompt tokens masked, sequences truncated to 1024 tokens, the model’s own tokenizer, and the same 64 measurement probes per capability for every state that is compared within a model. All responses are within-model differences on identical token sequences, in native nats per token: for pruning and quantization the damage $\Delta L_c = L_c(\text{compressed}) - L_c(\text{dense})$, and for distillation the signed transfer $\delta_c = L_c(S_{\text{KD}}) - L_c(S_0)$ of a student relative to its own initial state. Three boundaries apply throughout. The observable is the likelihood of a fixed reference, not a task-success measure; a QA loss can fall while exact-match accuracy does not rise. Native units differ across tokenizers, so absolute levels of different families are never placed on one scale. Capability specificity is a tested property rather than an assumption: on a pruning panel, the response on an independent secondary benchmark is predicted better from the same capability’s primary response than from another capability’s for math and QA, whereas for code no within-capability gain was detected at the available resolution (Appendix E).

2.2 Models and Compression Settings

Three kinds of model panel serve three roles (Table 1). A heterogeneous panel of finished public models establishes the phenomena and the limits of cross-family transfer. A controlled panel of Pythia checkpoints varies the initial model state along known axes, parameter count and pretraining stage, with architecture and data fixed; it is the development set of every

Panel (role)	Model states	Compression settings
Heterogeneous (phenomena, transfer limits)	12 finished models from 5 series: Qwen3 0.6B/1.7B/4B, Gemma-3 270M–27B, Gemma-4-31B, Muse-30B, OLMo-3 7B/32B	pruning at 10 densities in [0.3, 0.9]; RTN int8/6/5/4/3; Gemma-3 students distilled under two recipes
Controlled (development)	Pythia 160M, 410M, 1.4B at steps 16k, 64k, 143k (nine states)	pruning at $d \in \{0.9, 0.8, 0.7, 0.6\}$; RTN at $b \in \{8, 6, 4, 3\}$; grouped RTN at $b \in \{3, 5\} \times g \in \{64, 256\}$ on six of the states; LoRA distillation from one teacher pool
New states / configurations (validation)	160M, 410M, 1.4B at 96k; 1B at 32k/96k/112k; 6.9B at 32k/80k/112k; 410M at 48k; 1.4B at 112k; Gemma-3 270M/1B/4B students	densities 0.85/0.75/0.675/0.65/0.575/0.55; 5-bit; $b = 4$ and $g = 128$ grouped cells; distillation pools U225 and U375, 4B students

Table 1: Panels and their roles. Full checkpoint revisions, probe seeds, and hyperparameters are listed in Appendix A.

relation in §3. New states and configurations that entered no fit provide the validation of §4.

The three operations are held fixed by protocol. Pruning is global magnitude pruning of all transformer weight matrices to a retained density d . Quantization is symmetric round-to-nearest of the same matrices, weights only, either per output channel at bit-width b or with contiguous input groups of g weights sharing one scale. Distillation is supervised fine-tuning of an existing student on black-box teacher traces with strict LoRA adapters, where a pool of U traces per domain with D_U supervised completion tokens is processed for a fixed budget; we record the processed tokens, the supervised completion tokens T , and the reuse count $E = T/D_U$. The inputs admitted to every predictor are the parameter count N_0 of the transformer weight matrices, the pretraining token count D_0 where disclosed, and the dense capability losses $L_{0,c}$ measured once on the uncompressed model (the initial student for distillation). Every prediction is written as

$$\hat{L}_{m,c} = L_{0,c}^{\text{obs}} + \hat{\Delta}_{m,c}(\mathbf{x}, \boldsymbol{\theta}_m), \quad \mathbf{x} = (N_0, D_0, L_{0,c}), \quad (2)$$

with $\boldsymbol{\theta}_m$ the setting of method m (density, bit-width and group size, or pool and budget). The function $\hat{\Delta}_{m,c}$ is method-specific; the arms share the endpoint and the evaluation protocol, not a functional form.

2.3 Fitting and Evaluation

Each relation is fit on development states and settings only and is judged on two kinds of unseen target: an unseen setting (a density, bit-width, group size, pool, or budget outside the fitted set) and an unseen state (a checkpoint of a new size or pretraining stage, or a new student). On the development panel we use leave-one-stage-out and leave-one-source-out cross-validation; for new states the forms, coefficients, and predictions of every candidate are committed to the repository before the compressed outcome is measured, and a prediction keeps its status whatever the outcome. No prediction uses a measurement of the target compressed model; the dense anchor $L_{0,c}$ is the only measured input.

Error is the mean absolute error of $\hat{\Delta}$ in nats per token, per capability. Every candidate is compared with three baseline classes on exactly the same cells: source-free references (zero change, a per-capability constant, a strength-only curve, or the median development curve), same-input alternatives (the same $\mathbf{x}$ with a different functional structure), and the no- D_0 ablation. Where several baselines exist the strongest per capability is reported and that choice is marked as retrospective; bootstrap intervals are given only for the leave-one-out panels, with source states as the resampling unit. Freeze commits, the provenance code of each comparison, and the fold rules are in Appendix I.

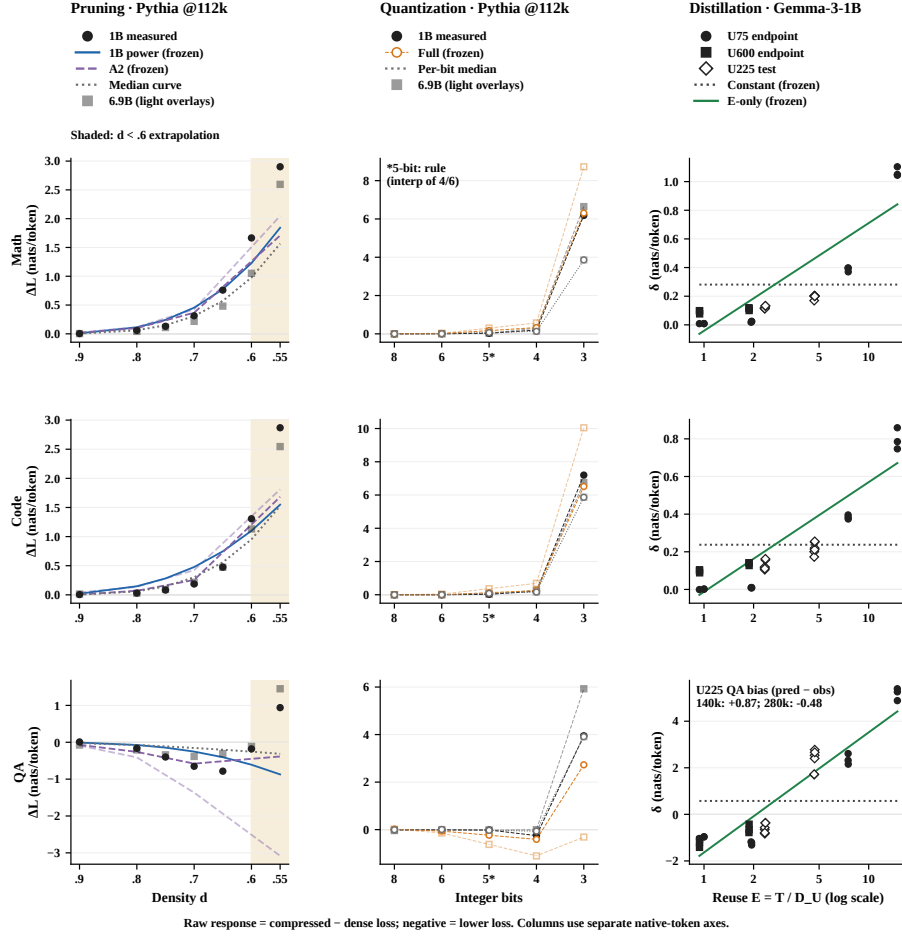


Figure 1: Capability-conditioned compression responses and frozen predictions, one source per arm. Left: pruning of Pythia-1B at step 112k (measured, filled circles) with the frozen shared power form (Eq. 3), the per-density regression A2, and the source-free median development curve; the 6.9B source at the same stage is overlaid in grey and the extrapolated density 0.55 is shaded. Middle: per-channel quantization of the same sources at the integer bit-widths with the frozen per-bit regression (hollow) and the per-bit median (dotted); the 5-bit point is the fixed interpolation rule and guide lines are visual only. Right: distillation of Gemma-3-1B, signed transfer δ_c against the reuse count E ; endpoint pools U75 and U600 (fit) and the unseen pool U225 (test) with the frozen constant and reuse-count forms and the QA bias at the two budgets. Points are measurements; lines are development fits frozen before the test points were measured. Native nats per token; separate axes per column.

3 Developing Capability-Conditioned Scaling Laws

3.1 Conditioning on the Initial Model State

The first question is whether the initial state is worth reading at all, and which part of it carries the signal. On the nine-state controlled panel, with the intervention setting fixed and one pretraining stage held out, a regression of the response on $(N_0, D_0, L_{0,c})$ at each setting lowers the pruning error from 0.413, 0.516, and 0.500 nats (the strongest source-free reference) to 0.253, 0.292, and 0.806 for math, code, and QA; the improvement interval excludes zero for code, includes zero for math, and is negative for QA, where predicting no change is better. Input ablations say which term matters (Fig. 3A). Adding the dense capability anchor $L_{0,c}$ to (N_0, D_0) has intervals above zero for math (+0.55, [0.29, 1.00])

Arm	Predictor	$P/\text{cap.}$	Varied setting; Target	Tested range	Coeff.
		dev. states	cal.		
Pruning	$A_c(\mathbf{x}) ((1 - d)/0.3)^{\gamma_c}$ (Eq. 3); same-input alternative A2 (per- d regression + interpolation)	5 (A2: 20)	d ; 17 Pythia states (160M–6.9B, 16k–143k)	0	$d \in [0.55, 0.9]$ Table 5
Quantization, fixed b	per-bit regression $\beta_{c,b} \cdot \phi(\mathbf{x})$	4 per b	new state at seen $b \in \{8, 6, 4, 3\}$; 9 dev states	0	interp. on new states; $d = 0.55$ extrap. fails seen b ; 5-bit register rule fails; near-zero regime un-informative
Quantization, (b, g)	$\phi(\mathbf{x}) \cdot [1, u, v, uv, u^2]$ (Eq. 4)	20	$b \in \{3, 5\} \times g \in \{64, 256\}$; 6 dev states	0	unseen $b = 4$, $g = 128$ on dev states; new state mixed
Distillation, source axis	$\delta_c = \beta_c \cdot \phi(\mathbf{x})$ (initial student)	4	9 Pythia students, pool/budget	0	held-out stage; register math only v39
Distillation, pool axis	$a_c + b_c \log E$ (and constant, $\log T$, joint)	2	Gemma-3-1B; pools U75/U600	0	unseen pool register U225: QA v41/v50 only; multi-student test frozen

Table 2: Predictive models and their domains. Inputs are always $\mathbf{x} = (N_0, D_0, L_{0,c})$ through $\phi(\mathbf{x}) = [1, \log \tilde{N}_0, \tilde{L}_{0,c}, \log \tilde{D}_0]$; no prediction uses a measurement of the compressed target (target calibration count 0). Ranges summarize §4; coefficients and standardization constants are in Appendix B.

and code (+0.66, [0.38, 1.13]); adding D_0 to $(N_0, L_{0,c})$ does not (+0.36, [−0.04, 0.60]; +0.40, [−0.01, 0.63]), and for quantization the D_0 increment clears zero in one capability (code) only. A frozen test at a new stage (step 96k on 160M and 1.4B, between the development stages) confirmed the same ranking: the full regression beat both its no- D_0 variant and the per-setting median on all three capabilities. We therefore condition every relation on the three inputs, read the dense anchor as the carrier of the signal, and keep D_0 as a weak second-order term. The anchor is used per capability; no joint capability vector is fitted, and nothing here shows that one would be needed.

3.2 Pruning

Response. Pruning damage grows with the removed fraction $1 - d$ and is source-dependent in amplitude: at the same density a late-stage small model can be several times more fragile than an early-stage one, and QA can improve while math and code degrade (Fig. 1, left). The relation to build is therefore a source-conditioned amplitude times a strength profile.

Functional form. The candidate we develop is

$$\widehat{\Delta L_c}(\mathbf{x}, d) = A_c(\mathbf{x}) \left(\frac{1 - d}{0.3} \right)^{\gamma_c}, \quad A_c(\mathbf{x}) = \beta_c \cdot (1, \log \tilde{N}_0, \tilde{L}_{0,c}, \log \tilde{D}_0), \quad (3)$$

five parameters per capability, where tildes denote standardization on the development set (Table 2 summarizes every delivered predictor). Its same-input alternatives are the per-density regression A2, which fits $\beta_{c,d}$ separately at each development density and predicts an unseen d by linear interpolation between the predicted anchors (four coefficients per density, twenty in total), the power form with γ_c fixed to one, and a continuous two-term polynomial in $1 - d$. The source-free references are a strength-only curve $A((1 - d)/0.3)^{\gamma_c}$, the median development curve, and zero change.

Parametric fit and selection. Fit on the nine development states at $d \in \{0.9, 0.8, 0.7, 0.6\}$ and evaluated at the pooled unseen densities 0.65 and 0.55, Eq. equation 3 cuts the error

of the strength-only curve by a factor of two to three on every capability, so reading the source state is what matters, while A2 is better on math and code and within noise on QA (Table 3), so the power structure adds nothing over the per-density regression. After the frozen tests of §4 we refit both on an expanded development set of 17 states including the unblinded 1B and 6.9B checkpoints and applied a pre-committed leave-one-source-out rule: A2 (0.479) and the power form (0.494) tie within 0.02 nats, and the rule prefers the continuous five-parameter form that vanishes at $d = 1$. Eq. equation 3 with the coefficients of Appendix B is the delivered pruning predictor; the exponent γ_c is a fitted constant per capability whose particular value is not established as necessary by these data.

3.3 Quantization

Per-bit source regression. Bit-width alone offers little room for a continuous relation: int8 and int6 stay within 0.01 nats of dense, int4 costs 0.03 to 0.35 nats on most states, and int3 collapses most models by several nats (Fig. 1, middle). The relation we develop at fixed bit-width is the source regression of §3.1 applied per bit, four coefficients per bit-width; it predicts a new state at a seen bit-width and is not a law over b . On the development panel it improves on the per-bit median for math and code with an interval above zero for code, and not for QA. The fixed shape $q_c 4^{-b}$ misfits this quantizer, whose integer range uses $q_{\max} = 2^{b-1} - 1$ levels per sign, and a learned exponential fits held-out bits better; neither is used as a predictor.

Group size as a second axis. A grouped quantizer whose scale is shared by g contiguous weights changes the error at a fixed bit-width and is known before the operation, so it adds a genuine configuration axis. On six development states at $b \in \{3, 5\}$ and $g \in \{64, 256\}$ the response has structure: 5-bit cells stay within 0.1 nats, 3-bit damage grows steeply with training stage (late 160M collapses by 15 to 17 nats, late 1.4B loses 0.6 to 2.2) and roughly halves when g drops from 256 to 64. Three relations were fit on those 24 cells with the same inputs $\mathbf{x}$: a separable form $A_c(\mathbf{x}) q_{\max}^{-p_c} (g/128)^{q_c}$ (six parameters), a low-order two-dimensional form

$$\widehat{\Delta L}_c(\mathbf{x}, b, g) = \phi(\mathbf{x}) \cdot [1, u, v, uv, u^2], \quad u = \log_2 q_{\max}(b) \text{ (centred)}, \quad v = \log_2 \frac{g}{128}, \quad (4)$$

with one four-vector of state coefficients per term (twenty parameters), and a same-input bilinear interpolation between the four development configurations. With only six states the leave-one-state-out errors are large for all three (the late 160M collapse dominates them), so the development set does not select a form; the tests of §4.1 do.

3.4 Distillation

Response and coordinates. The response is the signed transfer δ_c of an existing student to teacher traces, relative to the student’s own dense loss; the teacher and the size gap between teacher and student never enter it. Its coordinates are the supervised completion tokens T , the pool size D_U (the completion tokens of the U traces per domain), and the reuse count $E = T/D_U$; processed tokens and optimizer steps are recorded separately and never substituted for T . QA improves under every recipe we ran while math and code respond by a few hundredths to a few tenths of a nat, and with a small pool the QA loss rises again once E grows large (Fig. 1, right).

Source axis. On the nine Pythia states as students, with the teacher pool and budget fixed, a linear form $\delta_c = \beta_c \cdot (1, \log \tilde{N}_0, \tilde{L}_{0,c}, \log \tilde{D}_0)$ halves the error of a per-capability constant for math (0.014 against 0.031 with one stage held out), ties it for code (0.057 against 0.061), and is worse than a constant for QA (0.263 against 0.159). The student state therefore carries the math response and not the QA response.

Pool and budget axis. For a fixed student (Gemma-3-1B) we fit four forms on the endpoint pools U75 and U600: a constant, $a_c + b_c \log T$, $a_c + b_c \log E$, and $a_c + b_c \log T + c_c \log E$, all on the positive-budget range of those runs. Their independent test on an unseen pool is reported in §4.1. With the complete development matrix of the absolute-exposure protocol

(two students, two pools, three pool samples, four checkpoints) we also compared two forms that vanish at $T = 0$ and carry a student descriptor $z_c = \tilde{L}_{0,c}$,

$$\hat{\delta}_c = (a_c + \lambda_c z_c) \log(1 + E), \quad \hat{\delta}_c = (a_c + \lambda_c z_c)(1 - e^{-T/T_*}) + (b_c + \mu_c z_c) \log(1 + E), \quad (5)$$

by leave-one-run-out cross-validation: the first has the lowest math and code error (0.057 and 0.052 against 0.073 and 0.055 for $\log(1 + E)$ alone), the second the lowest QA error (0.706 against 0.798 for the reuse term alone and 1.40 for a constant). These are development results on two students; their predictions were specified after the frozen test matrix and are evaluated on it retrospectively (Appendix C). No joint student-pool-budget law is claimed.

4 Generalization to Unseen Models and Configurations

Once a form is chosen on the development set, its coefficients are frozen and it predicts three kinds of unseen object: compression settings outside the fitted set, model states of new sizes or pretraining stages, and, across both, the three capabilities. Table 3 lists every registered test with the candidate error and the strongest relevant baseline; Fig. 2 shows where the candidates gain over a source-free reference and where they do not. Errors are MAE in nats per token; “interpolation” and “extrapolation” refer to the fitted range of the setting.

4.1 Prediction at Unseen Compression Settings

Pruning densities. Inside the fitted density range the source-conditioned forms predict math and code and do not predict QA. On the three checkpoints that entered no fit, at densities between the grid points (0.85, 0.675, 0.575), the delivered power form has MAE 0.24, 0.24, and 0.68 for math, code, and QA: it beats the strength-only curve (0.45, 0.49, 0.58) and zero change on every checkpoint, has the lowest math error of all predictors, ties the median development curve on code, and loses QA to every source-free curve (median 0.22) because it over-predicts the QA improvement, so the source-free median has the lowest aggregate error (0.24 against 0.39). Beyond the fitted range the picture is uniform: at $d = 0.55$ every source-conditioned form fails on every source tested (errors 0.6 to 4.5 nats; the strength-only curve is closest), so the transfer range of these relations is the interpolation regime.

Quantization settings. The per-channel predictor has one unseen setting, 5 bits, and its fixed rule (the mean of the committed 4- and 6-bit predictions) is worse than a per-bit constant on the 1B source. With group size as a second axis the outcome changes. At the unseen bit-width $b = 4$ on the six development states, the two-dimensional form of Eq. equation 4 has MAE 0.19, 0.21, and 0.44 against 0.45, 0.53, and 0.48 for zero change and 0.55, 0.73, and 0.54 for the per-configuration median; at the unseen group size $g = 128$ across three bit-widths it has 0.22, 0.32, and 0.99 against 1.24, 1.35, and 1.24 for zero change, on all three capabilities. The separable candidate is worse than zero change on the bit test and the bilinear interpolation fails outside the development corners, so the response does not factor into a state amplitude and a shared shape. These are configuration transfers on seen states; the one new state tested (1B@96k at $g = 128$, three cells) is mixed, with the two-dimensional form best on code and behind the source-free curves on math and QA.

Distillation pools and budgets. The four forms frozen on the endpoint pools U75 and U600 were applied to an unseen pool U225 whose reuse count (2.3 to 4.7) lies inside the endpoint range and whose sampling protocol differs (random sample against pool prefix). The reuse-count form beats the constant only for QA (0.706 against 1.449); for math and code the constant is best (0.122 and 0.069 against 0.191 and 0.117) and the fitted forms over-extrapolate. The residual is dominated by systematic bias, -0.87 nats at one budget and $+0.48$ at the other for QA, and variation across pool samples exceeds variation across training seeds (0.43 against 0.04 nats). Under the absolute-exposure protocol, predictions of eight forms for three students on three unseen U375 pools were frozen before those runs began; their measurement was in progress at the data cut-off of this version and is not reported here.

Test	unseen	origin	math	code	QA
Source axis: seen setting, unseen source state					
prune: fixed d , 3×3 , leave-one-stage-out	stage	LFRA	0.253/0.413 _{med}	0.292/0.516 _{med}	0.806/0.500 ₀
quant: fixed b , 3×3 , leave-one-stage-out	stage	LFRA	1.144/1.410 _{med}	0.626/1.553 _{med}	1.989/1.473 _{med}
prune: stage 96k (160M, 1.4B), all d	stage	PFRA	0.090/0.343 _{nD}	0.128/0.179 _{nD}	0.426/0.438 _{nD}
quant: new stage 96k (160M/410M/1.4B), $b \geq 4$	stage	P-RA	0.029/0.084 _{pbm}	0.104/0.094 _{pbm}	0.173/0.134 _{pbm}
quant: new stage 96k (160M/410M/1.4B), int3	stage	P-RA	1.488/1.934 _{pbm}	1.963/0.843 _{pbm}	2.046/1.727 _{pbm}
distill: δ_c , 3×3 LoRA, leave-one-stage-out	stage	LFRA	0.014/0.026 _{nD}	0.057/0.044 _{nD}	0.263/0.159 _c
Configuration axis: unseen setting, size, stage, or pool					
prune: d 0.65/0.55, dev sources + 410M@96k	d	PRFA	0.371/0.283 _{A2}	0.524/0.279 _{A2}	0.759/0.818 _{A2}
prune: 1B@96k, $d = 0.65$	size, stage, d	PFFA	0.020/0.009 _{A1}	0.092/0.031 _{A1}	0.466/0.380 _{A2}
prune: 1B@96k, $d = 0.55$	size, stage, d	PFFA	0.930/0.273 _{so}	1.550/0.018 _{so}	1.426/0.530 _{so}
quant: 1B@96k, int4	size, stage	PFFA	0.116/0.068 _{pbm}	0.046/0.012 _{pbm}	0.059/0.054 _{nD}
quant: 1B@96k, int3	size, stage	PFFA	0.690/1.414 _{pbm}	0.318/0.001 _{pbm}	0.182/0.446 _{pbm}
prune: 1B@32k+112k, d 0.9–0.6	size, stage	PFFA	0.215/0.169 _{so}	0.191/0.119 _{A2}	0.261/0.227 _{med}
prune: 1B@32k+112k, $d = 0.55$	size, stage, d	PFFA	1.790/0.939 _{so}	1.488/0.089 _{so}	2.225/0.393 _{so}
quant: 1B@32k+112k, $b \geq 4$ (5-bit rule incl.)	size, stage, b	PFFA	0.064/0.020 _{pbm}	0.041/0.028 _{pbm}	0.157/0.052 _{pbm}
quant: 1B@32k+112k, int3	size, stage	PFFA	0.256/1.466 _{pba}	0.362/1.537 _{pba}	1.972/1.000 _{pbm}
prune: 6.9B@32k+112k, d 0.9–0.6	size $\uparrow$, stage	PFFA	0.299/0.120 _{med}	0.160/0.144 _{A2}	0.909/0.120 _{med}
prune: 6.9B@32k+112k, $d = 0.55$	size $\uparrow$, stage, d	PFFA	0.615/0.618 _{A2}	0.547/0.535 _{A2}	4.472/0.904 _{so}
quant: 6.9B@32k+112k, $b \geq 4$	size $\uparrow$, stage, b	PFFA	0.181/0.036 _{pbm}	0.177/0.043 _{pbm}	0.428/0.034 ₀
quant: 6.9B@32k+112k, int3	size $\uparrow$, stage	PFFA	3.233/2.046 _{pba}	3.229/1.805 _{pba}	5.306/1.744 _{pba}
distill: unseen pool U225 (Gemma-3-1B)	pool	PFFA	0.191/0.122 _c	0.117/0.069 _c	0.706/0.710 _{TE}

Table 3: Prediction tests, all three arms (MAE in nats/token per capability, candidate / strongest baseline, bold = better). The candidate is the source-conditioned predictor of each arm (configuration-indicator regression on the source axis and for quantization; shared power form for pruning on the configuration axis; E -only form for the distillation pool test). The baseline shown is the stronger of the same-input and the simple baseline per capability, named by code: nD = no- D_0 variant, med = per-configuration median or median development curve, 0 = zero change, so = strength-only curve, A1 = $\gamma=1$ power form, A2 = per-density regression with fixed interpolation, ct = continuous two-term form, pbm/pba = per-bit median/mean, c = constant, TE = joint $T+E$ form. “unseen” lists what the test holds out (size $\uparrow$ = five times beyond the development range). Origin code, four letters: prediction (P frozen prospective, L leave-one-out), same-input baseline, simple baseline (F pre-specified with the candidate, R computed after the test, – none), selection (A: all pre-specified forms reported). Improvements, both baselines separately, and bootstrap intervals are in Appendix Tables 12–14. Generated by v52_prediction_tables.py.

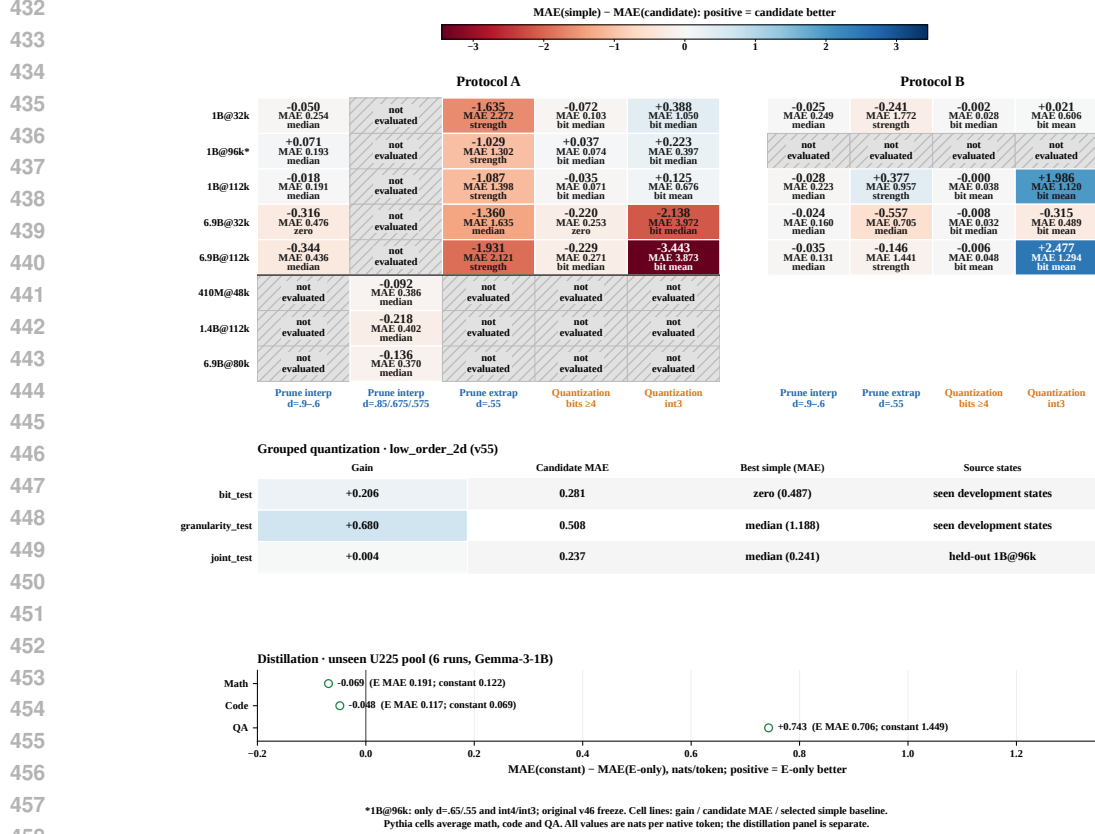


Figure 2: Generalization to unseen configurations and states. Top: improvement of the headline candidate over the strongest source-free reference (MAE difference, nats, averaged over the three capabilities; positive = candidate better) for each new source (rows), regime (columns), and fitting protocol (panels), with the candidate MAE printed in each cell; the confirmation panel (410M@48k, 1.4B@112k, 6.9B@80k) and the grouped-quantization tests are included. Cells not evaluated are hatched. Bottom: the distillation pool test, reuse-count form against the constant on the unseen pool U225, per capability. All predictions were frozen before the corresponding measurement.

4.2 Transfer across Model Sizes and Training Stages

Unseen stage inside the range. At step 96k, between the development stages, the frozen per-density pruning regression beat both its no- D_0 variant and the per-setting median on all three capabilities for the 160M and 1.4B sources (Table 3). The per-bit quantization regression on three sources at the same stage gains nothing over the per-bit median at four bits and above, where responses are near zero, and is inconsistent at int3 (better for math, worse for code and QA).

New size inside the range. For a 1B source at steps 32k and 112k, at interpolated densities, the power form, A2, and the source-free median curve are within 0.05 nats of one another on every capability; per stage, A2 is the best predictor at 112k on each capability and the median curve is best at 32k, driven by QA. A new in-range size is therefore predicted about as well by the source relation as by the development median.

A source five times larger. For a 6.9B source the source-conditioned pruning forms lose to the median curve on math (0.30 against 0.12) and QA (0.91 against 0.12) and tie on code: the regression extends the panel’s size trend into QA improvements of one to three nats that do not occur, and over-predicts math and code damage by about a factor of two at

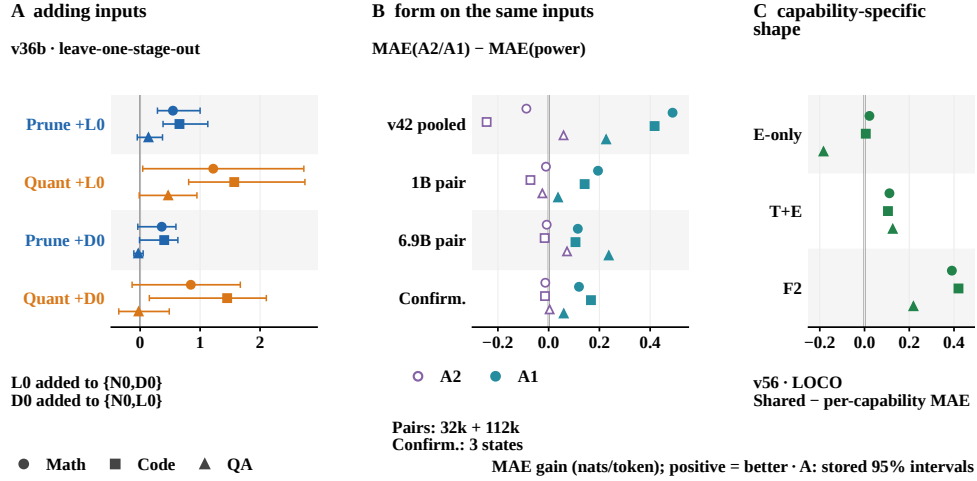


Figure 3: Which inputs and forms matter. A: improvement (nats, positive = better) from adding the dense anchor $L_{0,c}$ to (N_0, D_0) and from adding D_0 to $(N_0, L_{0,c})$ on the controlled panel, leave-one-stage-out, with 95% cluster-bootstrap intervals. B: improvement of the shared power form over the same-input alternatives A2 and A1 on every pruning test (positive = power form better). C: gain of per-capability response shapes over a shared shape at matched parameter count on the distillation development matrix (positive = per-capability better). Intervals only where computed.

mid densities. Fitting on early stages only (protocol B) brings the errors to the level of the source-free curves without beating them. The per-bit quantization regression is worse than the per-bit median or zero change at every bit-width for this source, by 0.1 to 0.4 nats above 3 bits and by 1.2 to 3.6 nats at int3, with one protocol-B exception at int3 (1.29 against 3.8 to 4.1). On the third new checkpoint of the confirmation panel, 6.9B@80k at interpolated densities, the delivered power form ties the median curve on math (0.30 against 0.31) and loses on code and QA, so the size boundary is soft for math and hard for QA.

New students. Predictions for Gemma-3-4B as a held-out student, at the development pools and at the unseen pools, are frozen and their measurement was in progress at the cut-off.

4.3 Applicability across Capabilities and Shared Structure

The same asymmetry recurs on every axis. Math and code are carried by the initial-state inputs: the dense anchor has interval-supported value for them, the pruning forms predict them at unseen densities and stages, the two-dimensional quantization form predicts them at unseen settings, and the distillation source form predicts math. QA is not: its pruning response is over-predicted by every source-conditioned form on every new checkpoint, a constant or a source-free curve is better for it on both distillation axes except the reuse-count form on the unseen pool, and its quantization error is the largest of the three on every test. Capability conditioning is also needed in the response shape, not only in the amplitude: on the complete distillation development matrix, per-capability models with the same parameter count as a shared-shape model lower the held-out error by 0.1 to 0.4 nats on every capability for the budget-plus-reuse structures, and only marginally for the reuse-only structure (Appendix C). Code results carry the caveat of §2.1: the within-capability specificity of the code loss was not detected on the secondary benchmark.

What is shared and what varies. We tested the stronger hypothesis that one function family serves all three methods with method- and capability-specific parameters. It holds for pruning by construction (the shared power family is the delivered form) and fails for the other two arms on the same held-out splits: a separable amplitude-times-shape family

is worse than the two-dimensional quantization form (macro MAE 2.96 against 2.40 on six held-out states), and a single reuse-count family is worse than the budget-plus-reuse distillation form (0.54 against 0.28, driven by QA). Within pruning, one exponent shared across capabilities costs nothing (leave-one-source MAE 0.351, 0.338, 0.785 against 0.328, 0.355, 0.798 per capability), whereas within quantization the term coefficients must be capability-specific (3.46 against 0.91 for math). Three ranges are kept apart (Appendix C): the bootstrap confidence intervals of the delivered pruning coefficients exclude zero for the dense-anchor term on math and code and not on QA; the exponents fitted separately per source spread from 3.0 to 6.0 for math and from 0.5 to 6.0 for QA, so the shape is only roughly stable across models; and prediction intervals built from leave-one-source residuals (per capability, 80%) cover 78% of the confirmation-panel cells and 67% of the 1B and 6.9B pair cells at a mean full width of about 1.5 nats, which is honest but wide relative to most responses.

One calibration point. Setting the amplitude of the pruning form from one measured compression point of the target, its mildest density, and predicting the rest with the shared exponent is not a usable law: the mildest responses are within noise and a steep exponent amplifies that noise (leave-one-source MAE above 17 nats; on the confirmation panel calibrated at $d = 0.85, 0.16, 0.51, 0.58$ against 0.36, 0.33, 0.99 for the uncalibrated form and 0.41, 0.32, 0.32 for the median curve). For grouped quantization the opposite holds: a separable law with its amplitude set at the single 5-bit cell $g = 256$ predicts the unseen 4-bit configurations on states held out of the fit with MAE 0.13, 0.12, 0.29, against 0.26, 0.64, 1.34 for the two-dimensional form without calibration, at the cost of one compressed measurement per source. Which calibration is worth its cost is therefore method-specific.

5 From Laws to Capability-Specific Selection

The laws of §3 are meant to be used: given an initial model, a capability, and a resource constraint, which method and setting preserves that capability best? We write the decision as

$$(m^*, \theta_m^*) = \arg \min_{m, \theta_m \in \mathcal{F}_m(B)} \hat{L}_{m,c}(\mathbf{x}, \theta_m), \quad (6)$$

where $\mathcal{F}_m(B)$ is the set of settings of method m that meet the budget B . Methods are compared through their predicted capability loss and their resource cost, not through their internal equations, and the map for one capability need not coincide with the map for another.

We validated this use on the controlled panel with a nominal storage budget, using only measured configurations (84 pruning, 70 per-channel and 57 grouped quantization, and 9 same-stage smaller students across 17 states) and refitting every law with the target state held out, so that no prediction sees the state it selects for. Over 289 state-budget cells per capability, the choice made from the laws is within 0.19 (math), 0.17 (code), and 0.42 (QA) nats of the measured oracle on average, and within 0.03 nats when the objective is the worst of the three capabilities; it picks the oracle’s method in 96%, 95%, 73%, and 94% of cells. The cheapest feasible configuration loses 4.3 to 5.2 nats to the oracle. A fixed quantization-only policy is as good as the map on math and code, because on this storage axis quantization dominates those capabilities for most states (Fig. 4); the map departs from it where the laws predict otherwise, for QA on the 410M and 1.4B states that have a smaller student, where distillation is chosen, and on the 1B and 6.9B states whose QA improves under pruning, where pruning is chosen. The fixed pruning-only and distillation-only policies cannot meet the budget in 44% and 63% of cells. About half the cells are “no clear winner” cells, in which the predicted gap between the two best methods is smaller than the laws’ held-out error; reporting the candidate set there is the honest output, and the oracle’s method lies in that set in every such cell (Appendix Table 20). The same pattern appeared earlier in an offline replay on twelve heterogeneous models (Appendix H). These maps inherit the laws’ boundaries: they are validated on in-range states with a nominal storage cost, without latency or the cost of recovery training, and their QA cells are the least reliable.

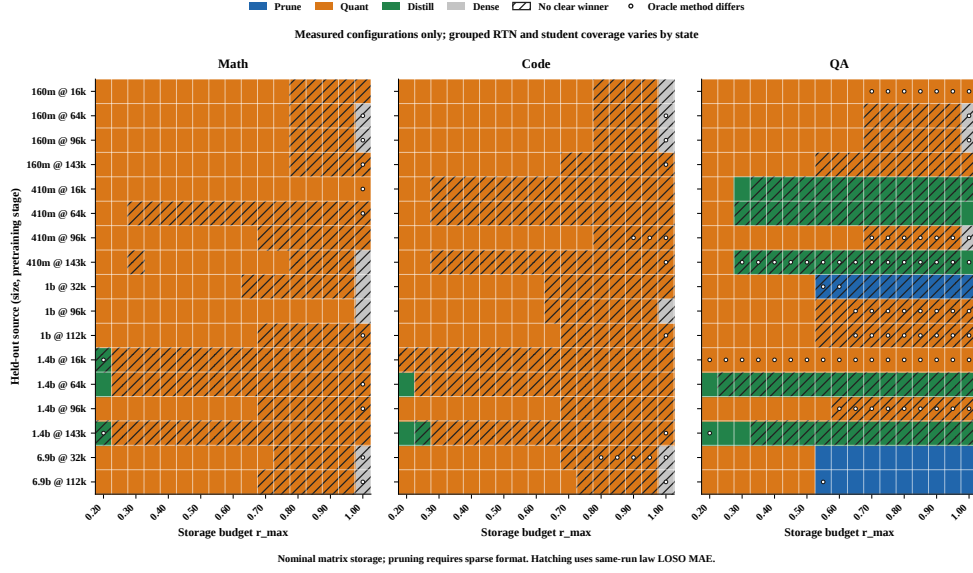


Figure 4: Capability-specific selection maps on held-out Pythia states. For each held-out state (rows, ordered by size and stage) and nominal storage budget $r_{\max}$ (columns), the colour is the method whose law-predicted loss is lowest among the measured configurations within budget (pruning $r = d$, per-channel RTN $r = b/16$, grouped RTN $r = (b + 16/g)/16$, distillation to a smaller same-stage student $r = N_S/N_0$; the dense source at $r = 1$); every law is refit with the state held out. Hatching marks cells in which the predicted gap between the two best methods is smaller than the larger of their leave-one-state-out errors; a marker shows where the measured oracle chooses a different method. Nominal pruning storage requires a sparse format. Generated by `v60_selection_maps.py`.

6 Related Work

Scaling laws for compression. Existing compression laws differ in what they take as input and what they predict. Sparsity enters as a multiplicative factor on the size term of a pretraining loss law (Frantar et al., 2024); precision enters through an effective-parameter count with a post-training degradation term that grows with pretraining tokens (Kumar et al., 2025); formats used during training collapse onto one capacity scalar (Panferov et al., 2025); the P^2 law predicts the post-training loss of a pruned model from its size and loss before pruning, the pruning rate, and the post-training tokens, validated on held-out tokens, sizes, and rates (Chen et al., 2025); and a distilled student follows a broken power law in teacher loss during pretraining-scale logit distillation (Busbridge et al., 2025). Their targets are aggregate losses; the P^2 law shows that initial-state inputs are established practice, and we follow it in admitting the pre-compression loss. What we add is a per-capability endpoint measured the same way for three methods, a development panel in which size and pretraining stage vary separately, and tests that hold out settings and states with baselines on the same inputs.

Task-conditioned evaluation of compression. Pruning disproportionately harms knowledge-intensive tasks (Jaiswal et al., 2024), low-bit quantization disproportionately harms reasoning (Liu et al., 2025), and selective truncation can improve specific behaviors (Sharma et al., 2024). Laws with task-level targets exist: Sengupta et al. (2025) fit pruning laws to test loss and averaged zero-shot accuracy against the pruning ratio and recovery tokens, and their revised version adds rolling hold-out over ratios, zero- and one-shot transfer to unseen models, and per-task exponents, with the unpruned score and the ratio as the only inputs; Zhou et al. (2026) stratify post-training quantization laws into memorization, application,

and reasoning strata for one quantizer, validated on a held-out larger model; task-specific distillation has been mapped as curves over dataset size, compression ratio, and supervision format (Ghita et al., 2026). Relative to these, our relations use a continuous conditional loss rather than accuracy, admit the dense capability anchor and training tokens as inputs, and treat the three methods with one protocol.

Fitting and validating empirical laws. Downstream accuracy is predictable mainly through continuous log-likelihood measurements (Schaeffer et al., 2023; 2025; Gadre et al., 2025; Bhagia et al., 2025), and linear loss-to-downstream relations hold in a minority of settings (Lourie et al., 2025), which motivates our loss endpoint and its stated boundaries. Fitted scaling-law forms are fragile to the fitting protocol (Li et al., 2025; Besiroglu et al., 2024), and data-constrained pretraining shows that repeated tokens are not worth new tokens (Muennighoff et al., 2023), an effect our reuse-count coordinate measures in the distillation setting without importing those coefficients. Curvature- and gradient-based importance underlies modern pruning (LeCun et al., 1989; Frantar & Alistarh, 2023; Sun et al., 2024); we use capability-conditional gradient statistics only as mechanism evidence (Appendix G), never as prediction inputs.

7 Discussion and Conclusion

What can be predicted now is specific. Given the parameter count, the pretraining token count, and the dense capability losses of a source in the development range, the pruning form of Eq. equation 3 predicts the math and code response at any density inside the fitted range to within a few tenths of a nat, and a per-density regression does the same without the power structure. Given the same inputs, the two-dimensional form of Eq. equation 4 predicts the response of the grouped quantizer at bit-widths and group sizes it was not fit on, for the states it was fit on. Given a student’s state, a linear form predicts its math transfer; given a fixed student, a reuse-count term predicts the QA transfer on a new pool with a bias that changes sign across budgets. Each of these comes with coefficients (Appendix B) and a tested range, and together they select methods and settings for held-out states within 0.2 nats of the measured optimum for math and code under a storage budget (§5).

What cannot be predicted is equally specific. No source-conditioned form predicts the QA response to pruning on a new checkpoint; a source-free curve is the better deliverable for that capability. No form transfers to density 0.55 or to a source five times larger than the development range, except that the pruning math error at the larger source is on par with the source-free curve. Bit-width alone supports no continuous quantization relation, and the separable amplitude-times-shape candidate fails once group size is added. The distillation relations are validated on one student per axis at the cut-off of this version; the frozen multi-student test is pending.

For further development these results say three things. The dense capability anchor is the input to keep and training tokens add little beyond it, so cheaper metadata-only prediction is not yet available. The functional forms that work are low-dimensional and the particular exponents are not established as necessary, so the next question is which inputs carry the QA response rather than which exponent to fix. Transfer across sizes is bounded by the development range, so a wider development panel is the direct route to wider laws. The limitations are those of small controlled panels with correlated observations (three sizes, three stages, three pools), of one quantizer family and one teacher, of a loss endpoint whose code specificity was not detected on a secondary benchmark, and of a selection validation that uses nominal storage as the only cost, without latency, recovery-training cost, or any agent-level measurement.

References

Tamay Besiroglu, Ege Erdil, Matthew Barnett, and Josh You. Chinchilla scaling: A replication attempt. arXiv preprint arXiv:2404.10102, 2024.

- Akshita Bhagia, Jiacheng Liu, Alexander Wettig, David Heineman, Oyvind Tafjord, Ananya Harsh Jha, Luca Soldaini, Noah A. Smith, Dirk Groeneveld, Pang Wei Koh, Jesse Dodge, and Hannaneh Hajishirzi. Establishing task scaling laws via compute-efficient model ladders. In Conference on Language Modeling (COLM), 2025.
- Dan Busbridge, Amitis Shidani, Floris Weers, Jason Ramapuram, Etai Littwin, and Russ Webb. Distillation scaling laws. In International Conference on Machine Learning, 2025.
- Xiaodong Chen, Yuxuan Hu, Xiaokang Zhang, Yanling Wang, Cuiping Li, Hong Chen, and Jing Zhang. P² law: Scaling law for post-training after model pruning. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, 2025.
- Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. In International Conference on Machine Learning, 2023.
- Elias Frantar, Carlos Riquelme, Neil Houlsby, Dan Alistarh, and Utku Evci. Scaling laws for sparsely-connected foundation models. In International Conference on Learning Representations, 2024.
- Samir Yitzhak Gadre, Georgios Smyrnis, Vaishaal Shankar, Suchin Gururangan, Mitchell Wortsman, Rulin Shao, Jean Mercat, Alex Fang, Jeffrey Li, Sedrick Keh, Rui Xin, Marianna Nezhurina, Igor Vasiljevic, Luca Soldaini, Jenia Jitsev, Alex Dimakis, Gabriel Ilharco, Pang Wei Koh, Shuran Song, Thomas Kollar, Yair Carmon, Achal Dave, Reinhard Heckel, Niklas Muennighoff, and Ludwig Schmidt. Language models scale reliably with over-training and on downstream tasks. In International Conference on Learning Representations, 2025.
- Andrei Ghita, Nirav Desai, and Alex Boier. Scaling laws for task-specific LLM distillation. arXiv preprint arXiv:2606.24747, 2026.
- Ajay Jaiswal, Zhe Gan, Xianzhi Du, Bowen Zhang, Zhangyang Wang, and Yinfei Yang. Compressing llms: The truth is rarely pure and never simple. In International Conference on Learning Representations, 2024.
- Tanishq Kumar, Zachary Ankner, Benjamin F. Spector, Blake Bordelon, Niklas Muennighoff, Mansheej Paul, Cengiz Pehlevan, Christopher Ré, and Aditi Raghunathan. Scaling laws for precision. In International Conference on Learning Representations, 2025.
- Yann LeCun, John S. Denker, and Sara A. Solla. Optimal brain damage. In Advances in Neural Information Processing Systems, 1989.
- Margaret Li, Sneha Kudugunta, and Luke Zettlemoyer. (mis)fitting: A survey of scaling laws. In International Conference on Learning Representations, 2025.
- Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. Quantization hurts reasoning? an empirical study on quantized reasoning models. In Conference on Language Modeling (COLM), 2025.
- Nicholas Lourie, Michael Y. Hu, and Kyunghyun Cho. Scaling laws are unreliable for downstream tasks: A reality check. In Findings of the Association for Computational Linguistics: EMNLP 2025, 2025.
- Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In Advances in Neural Information Processing Systems, 2023. arXiv:2305.16264.
- Andrei Panferov, Alexandra Volkova, Ionut-Vlad Modoranu, Vage Egiazarian, Mher Safaryan, and Dan Alistarh. Unified scaling laws for compressed representations. In Advances in Neural Information Processing Systems, 2025.
- Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? In Advances in Neural Information Processing Systems, 2023.

Rylan Schaeffer, Hailey Schoelkopf, Brando Miranda, Gabriel Mukobi, Varun Madan, Adam Ibrahim, Herbie Bradley, Stella Biderman, and Sanmi Koyejo. Why has predicting downstream capabilities of frontier ai models with scale remained elusive? In International Conference on Machine Learning, 2025.

Ayan Sengupta, Siddhant Chaudhary, and Tanmoy Chakraborty. Pruning laws for large language models. arXiv preprint arXiv:2504.04342, 2025. v2, EMNLP 2026; v1 (2025) was titled Compression Laws for Large Language Models.

Pratyusha Sharma, Jordan T. Ash, and Dipendra Misra. The truth is in there: Improving reasoning in language models with layer-selective rank reduction. In International Conference on Learning Representations, 2024.

Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. In International Conference on Learning Representations, 2024.

Chenxi Zhou, Pengfei Cao, Jiang Li, Bohan Yu, Jinyu Ye, Jun Zhao, and Kang Liu. Task-stratified knowledge scaling laws for post-training quantized large language models. In Findings of the Association for Computational Linguistics: ACL 2026, 2026.

A Experimental Details

A.1 Probes and loss computation

Each capability is measured on a fixed probe set of prompt-reference pairs drawn once, with a fixed seed, from held-out benchmark splits that are disjoint from each training and recovery source used in the paper. The primary measurement benchmarks are MATH-500 for mathematical reasoning, MBPP for code generation, and 2WikiMultihopQA for multi-hop question answering. For each capability we draw 128 probes and split them by index parity into an estimation half, used only for gradient and Fisher statistics, and a measurement half of 64 probes on which each reported loss is computed, so that mechanism statistics and evaluations do not share sampling noise. Secondary benchmarks used for the measurement checks below are GSM8K, HumanEval, and HotpotQA on the pruning panel (no training involved) and SVAMP, HumanEval, and TriviaQA where distilled models are scored, chosen to exclude the benchmarks from which the distillation traces were derived; a sample-level audit found no overlap between any measurement probe and any training pool. The measurement half is 64 probes per capability for every result in the paper except where a 512-probe re-measurement is stated.

A.2 Compression protocols

Pruning. Global magnitude pruning removes the smallest weights across all transformer matrices (attention and MLP projections; embeddings, output head, biases, and norms are excluded) to a target retained density d . The global threshold is estimated from a seeded sample of absolute weights so that the same procedure scales to 30B-parameter models; each density is applied to the same dense reference weights, which are held on the CPU for the largest models so that a single 80 GB device suffices.

Quantization. Weights are fake-quantized per output channel with symmetric round-to-nearest at bit-width b , using the integer range $[-(2^{b-1} - 1), 2^{b-1} - 1]$ and a per-channel scale set by the channel’s maximum absolute value; activations are left in bfloat16. The dense weights are restored between bit-widths. The quantizer is fake_quantize_per_output_channel in the released code: $q_{\max} = 2^{b-1} - 1$ is the largest positive integer, integers are clamped to $[-q_{\max}, q_{\max}]$ ($2^b - 1$ values), and the scale is the channel’s maximum absolute weight divided by $q_{\max}$. With a fixed clipping range the step size from 4 to 5 bits shrinks by 7/15, so the squared step ratio is $(7/15)^2 \approx 0.22$; in general the squared step ratio between adjacent bit-widths is $(q_{\max}(b)/q_{\max}(b+1))^2$, not a constant

1/4. A squared step ratio becomes a loss ratio only under assumptions about the error distribution and the local response that we do not test, so the fixed 4^{-b} shape is one candidate among others rather than a derived reference. Nominal storage ratios are computed from bit-widths; no memory or latency was measured.

Distillation. Students are fine-tuned on black-box teacher traces (a frontier API teacher answering GSM8K, HotpotQA, and CodeAlpaca prompts; 600 traces per domain) under the full-trace recipe, with strict LoRA adapters of rank 16 and scaling 32 on the query, key, value, output, gate, up, and down projections, AdamW at learning rate 10^{-4} , an effective batch of 16 sequences, and a cosine schedule with 3% warmup. A run aborts if the adapter library is unavailable and does not fall back to full fine-tuning silently, and the actual training mode and trainable parameter count are recorded in each manifest. Token accounting distinguishes processed tokens (all input tokens seen, including repetitions), unique pool tokens, and supervised completion tokens. The data pool of U examples per domain is either the first U rows of the trace file (the historical protocol) or a random sample drawn with a pool-sampling seed that is independent of the training seed; pools are identified by the sorted hashes of their samples and pairwise overlaps are reported. The historical schedule spanned a fixed number of epochs, so its cosine horizon varied with pool size; the controlled matrix of §3.4 uses a schedule defined on the same absolute processed-token budget for each pool, and the two protocols are kept apart in fitting.

Grouped quantizer. The grouped variant (§3.3) splits each row of a weight matrix into contiguous groups of g input weights, pads the last group with zeros (which do not affect the group’s absmax), and applies the same symmetric round-to-nearest with $q_{\max} = 2^{b-1} - 1$ per group; g equal to the row width reproduces the per-channel quantizer bit-exactly (verified by a unit test). Configurations are $b \in \{3, 4, 5\}$ and $g \in \{64, 128, 256\}$; nominal storage is computed from b and g , and no memory or latency was measured.

A.3 The controlled Pythia panel

The panel uses the public Pythia checkpoints at sizes 160M, 410M, and 1.4B and pretraining steps 16,000, 64,000, and 143,000, which correspond to 33.6B, 134.2B, and 299.9B processed tokens at the published batch of 1024×2048 tokens per step. The size coordinate N_0 counts transformer-matrix parameters only (the query, key, value, output, and MLP matrices of each layer), computed from the architecture configuration, and the training-tokens coordinate is $D_0 = \text{step} \times 2,097,152$. Each checkpoint’s weight identity was verified before use: the 2.8B revisions published for different steps were found to load identical weights and were excluded, and an integrity guard now rejects any pair of nominally different checkpoints with identical measured panels. New sources added for the prospective tests (Pythia-1B at steps 32,000, 96,000, and 112,000; Pythia-6.9B at 32,000 and 112,000) were confirmed to have not entered any fit or model selection before their predictions were frozen.

The three checkpoints of the confirmation panel (410M@48k, 1.4B@112k, 6.9B@80k) were verified as distinct weights before use: their model-shard hashes differ from every other cached revision of the same size, and their dense capability vectors differ from those of the neighbouring stages.

Cohort	Sources	Interventions	Probes	Role
Heterogeneous panel	12 dense checkpoints, 5 series from 4 families: Qwen3 0.6B/1.7B/4B, Gemma-3 270M–27B, Gemma-4-31B, Muse-30B, OLMo-3 7B/32B (+2 prospective: Qwen3 8B/14B)	global magnitude pruning, 10 densities in [0.3, 0.9]; RTN int8/6/5/4/3	64 per capability, primary benchmarks	phenomena; cross-family transfer; noise screen
Controlled Pythia panel	sizes 160M/410M/1.4B $\times$ steps 16k/64k/143k (9 states); new sources 160M, 410M, 1.4B at 96k; 1B at 32k/96k/112k; 6.9B at 32k/112k	pruning: 4 dev densities 0.9–0.6, tests at 0.75/0.65/0.55; bits 8/6/4/3, test 5	same	development set; source-axis and configuration-axis tests
Distillation, controlled	the 9 Pythia states as students; fixed teacher pool (600 traces per domain); LoRA	fixed recipe	same	δ_c source transfer
Distillation, pools	Gemma-3-1B student; pools U75 and U600 (fit), U225 (test); 3 pool seeds $\times$ 2 training seeds	processed tokens T ; reuse count $E = T/D_U$	same	configuration axis (pool)
Distillation, heterogeneous	Gemma-3 270M–12B, OLMo-3-7B, Qwen3 0.6B–4B students; two API teachers; full-trace vs answer-only	recipe	same	phenomena only

Table 4: Cohorts and their roles. Counts are generated from the result files (v51_panel_tables.py); the two Qwen3 prospectives were measured at four densities only. The 2.8B Pythia checkpoints were excluded because the step revisions we downloaded loaded identical weights (hash check); nothing is claimed about other revisions.

B Fitted Models and Coefficients

The delivered pruning form (Eq. equation 3) and its standardization are in Table 5; the grouped-quantization form (Eq. equation 4) in Table 6; the per-bit source regressions, the distillation source form and pool forms, and the frozen multi-student forms are stored with their standardization constants in the register files listed in Appendix I.

capability	γ_c	$\beta_{c,0}$	$\beta_{c,\log N_0}$	β_{c,L_0}	$\beta_{c,\log D_0}$
math	3.75	+3.275	+0.319	+3.850	+0.250
code	3.05	+4.499	+0.273	+5.357	+0.347
qa	3.05	-7.409	-0.481	+5.486	+0.106

Table 5: Selected pruning form $\widehat{\Delta L_c} = (\beta_c \cdot \phi)((1 - d)/0.3)^{\gamma_c}$ with $\phi = [1, z(\log N_0), z(L_{0,c}), z(\log D_0)]$; standardization centers 20.248, 2.661, 25.614 and scales 1.407, 1.691, 0.739 for $(\log N_0, L_{0,c}, \log D_0)$ in natural units (N_0 = transformer weight-matrix parameters, D_0 = tokens). Register: v53-prune-dev/register.json.

capability	term	β_0	$\beta_{\log N_0}$	β_{L_0}	$\beta_{\log D_0}$
math	1	+5.664	+0.569	+7.260	+0.577
math	u	-13.029	-1.306	-16.698	-1.326
math	v	+0.297	-0.026	+0.117	+0.152
math	uv	-0.162	+0.030	+0.017	-0.119
math	u^2	+7.634	+0.767	+9.786	+0.777
code	1	+5.304	+0.349	+6.539	+0.534
code	u	-12.130	-0.796	-14.945	-1.223
code	v	+0.753	-0.006	+0.619	+0.231
code	uv	-0.560	+0.011	-0.425	-0.188
code	u^2	+7.149	+0.470	+8.813	+0.720
qa	1	-9.723	-0.684	+7.403	+0.141
qa	u	+21.703	+1.561	-16.563	-0.359
qa	v	+1.825	-0.011	-1.220	+0.126
qa	uv	-1.561	+0.022	+1.028	-0.125
qa	u^2	-13.105	-0.922	+9.978	+0.190

Table 6: Grouped-quantization form $\widehat{\Delta L}_c = \sum_k \beta_{c,k} \cdot \phi(\mathbf{x}) t_k(b, g)$ with terms $t_k \in \{1, u, v, uv, u^2\}$, $u = \log_2 q_{\max}(b)$ centred at the development mean, $v = \log_2(g/128)$; standardization centers 19.565, 2.746, 25.332 and scales 1.084, 1.623, 1.095 for $(\log N_0, L_{0,c}, \log D_0)$; ridge 10^{-3} . Register: v55-quant-group/register.json.

C Alternative Functional Forms and Model Selection

C.1 Pruning: expanded development set and the selection rule

subset	candidate	math	code	QA	mean
all	power	0.328	0.355	0.798	0.494
all	A1	0.610	0.608	0.905	0.708
all	A2	0.345	0.295	0.796	0.479
all	cont	0.440	0.394	0.828	0.554
all	strength only	0.492	0.601	0.692	0.595
all	median curve	0.410	0.455	0.540	0.468
all	zero	0.812	0.852	0.602	0.755
off coarse	power	0.592	0.649	1.370	0.870
off coarse	A1	1.090	1.068	1.343	1.167
off coarse	A2	0.959	0.704	1.459	1.041
off coarse	cont	0.758	0.638	1.306	0.900
off coarse	strength only	0.909	1.077	1.163	1.050
off coarse	median curve	0.758	0.790	0.901	0.816
off coarse	zero	1.715	1.725	1.090	1.510

Table 7: Pruning development on 17 Pythia states: leave-one-source-out MAE (nats) over all held-out rows (‘all’) and over the densities off the coarse grid (0.75, 0.65, 0.55). Pre-committed rule: lowest mean over capabilities among the four source-conditioned candidates; a gap below 0.02 nats between the best two is resolved toward the continuous, zero-at- $d=1$ form with fewer parameters, which selected the power form over A2 (gap 0.015). Baselines do not select.

C.2 Grouped quantization: leave-one-state-out on six states

candidate	math	code	QA
separable	1.953	2.185	4.749
low order 2d	0.914	1.910	4.371
same input interpolation	1.230	1.889	4.333
mean	2.623	2.566	2.891
median	1.694	1.758	1.777
zero	1.821	1.929	1.796

Table 8: Grouped quantization: leave-one-state-out MAE (nats) on the 24 development cells (six states). The late-stage 160M state, which collapses at 3 bits, dominates every held-out error, so the development set does not rank the forms; all three candidates and the baselines were frozen and tested (§4.1).

C.3 Distillation: descriptor forms and capability conditioning

Form	P	LOCO math	LOCO code	LOCO qa	LOSO math	LOSO code	LOSO qa
zero	0	0.152/-0.152	0.178/-0.178	1.429/+0.143	0.152/-0.152	0.178/-0.178	1.429/+0.143
constant	3	0.095/+0.000	0.103/+0.000	1.400/+0.000	0.093/+0.000	0.100/+0.000	1.358/-0.000
T-only	6	0.082/+0.000	0.072/+0.000	1.092/-0.000	0.089/+0.000	0.070/+0.000	1.075/-0.000
E-only	6	0.073/-0.003	0.055/-0.002	0.798/-0.029	0.091/+0.000	0.056/+0.000	0.949/+0.000
surface:L0	12	0.072/+0.000	0.054/-0.000	0.812/+0.001	0.091/+0.000	0.054/+0.000	0.944/-0.000
surface:logN	12	0.072/+0.000	0.054/-0.000	0.812/+0.001	0.091/+0.000	0.054/+0.000	0.944/+0.000
F1:L0	6	0.057/+0.003	0.052/+0.003	1.518/+0.662	0.152/-0.152	0.178/-0.178	1.429/+0.143
F1:logN	6	0.057/+0.003	0.052/+0.003	1.518/+0.662	0.152/-0.152	0.178/-0.178	1.429/+0.143
F2:L0	12	0.062/-0.007	0.057/-0.005	0.706/-0.069	0.152/-0.152	0.178/-0.178	1.429/+0.143
F2:logN	12	0.062/-0.007	0.057/-0.005	0.706/-0.069	0.152/-0.152	0.178/-0.178	1.429/+0.143

Table 9: Distillation development matrix (12 runs, 48 points): held-out MAE/signed bias (nats) per form and capability under leave-one-run-out (LOCO) and leave-one-student-out (LOSO). Forms: $t = \log(1 + T_c/35000)$, $e = \log(1 + E)$, $s = 1 - e^{-T_c/T_*}$; T-only $a + bt$; E-only $a + be$; surface $a + bt + ce + dz$; F1 $(a + \lambda z)e$; F2 $(a + \lambda z)s + (b + \mu z)e$, $z = \text{standardized } L_{0,c}$ (or $\log N_S$, identical up to sign with two students). Under LOSO the descriptor forms degenerate (one student in training) and fall back to zero.

Structure	P shared/per	Capability	Shared MAE	Per-cap MAE	Gain	>0.02
E-only	4/4	math	0.431	0.409	0.022	yes
E-only	4/4	code	0.417	0.412	0.006	no
E-only	4/4	qa	0.967	1.150	-0.183	no
E-only	4/4	macro	0.605	0.657	-0.052	no
T+E	5/5	math	0.439	0.327	0.111	yes
T+E	5/5	code	0.419	0.314	0.105	yes
T+E	5/5	qa	0.979	0.853	0.126	yes
T+E	5/5	macro	0.612	0.498	0.114	yes
F2	7/7	math	0.536	0.145	0.390	yes
F2	7/7	code	0.527	0.107	0.420	yes
F2	7/7	qa	1.044	0.826	0.219	yes
F2	7/7	macro	0.702	0.359	0.343	yes

Table 10: Capability conditioning at matched parameter count (leave-one-run-out MAE, nats): shared response shape with per-capability offsets versus per-capability models with the same total count; gain > 0.02 nats is a descriptive threshold.

C.4 Shared structure, parameter ranges, and one-point calibration

Table 11 reports the shared-family test, the parameter-sharing tests, the three kinds of range (bootstrap confidence intervals of the delivered coefficients; the spread of exponents fitted separately per source; prediction intervals from leave-one-source residuals with their

coverage on cells not used to build them), and the one-point calibration (K1) against the uncalibrated predictor (K0), generated by v59_shared_structure.py; K1 costs one compressed measurement per source.

Law	What is shared	What varies	Held-out shared vs specific	MAE K0 vs MAE K1	80% PI coverage & width
Pruning power	Power family	β_c, γ_c	0.494 vs 0.494	—	—
Grouped RTN	Separable family	β_c, p_c, q_c	2.962 vs 2.398	—	—
Distill exposure	$\log(1 + E)$ family	β_c ; inactive D_0	0.542 vs 0.275	—	—
Pruning sharing	One γ	β_c	0.491 vs 0.494	—	—
Quantization sharing	2D term vectors	Capability off-sets	3.999 vs 2.398	—	—
Prune math	Power family	Source amplitudes, γ	γ min/med/max 3.00/4.60/6.00	—	—
Prune code	Power family	Source amplitudes, γ	γ min/med/max 2.30/5.95/6.00	—	—
Prune qa	Power family	Source amplitudes, γ	γ min/med/max 0.50/3.40/6.00	—	—
Prune dev LOSO	Training γ_c	One target amplitude	—	0.604 vs 31.406	—
Prune confirmation	Training γ_c	One target amplitude	—	0.560 vs 0.415	77.8%/1.621 to 100.0%/3.629
Prune pairs: 17-source OOF	Training γ_c	One target amplitude	—	0.517 vs 0.763	—
Prune pairs: independent 13	Training γ_c	One target amplitude	—	0.707 vs 0.772	66.7%/1.523 to 87.5%/4.277
Quant dev LOSO	2D Training curve	One target amplitude	—	3.143 vs 26.177	—
Quant bit_test	2D Training curve	One target amplitude	—	0.747 vs 6.442	—
Quant granularity_test	2D Training curve	One target amplitude	—	1.751 vs 14.973	—
Quant joint_test	2D Training curve	One target amplitude	—	0.344 vs 0.193	—
Quant bit_test	sep Training curve	One target amplitude	—	1.540 vs 0.183	—
Quant granularity_test	sep Training curve	One target amplitude	—	2.423 vs 0.623	—
Quant joint_test	sep Training curve	One target amplitude	—	0.724 vs 0.306	—

Table 11: Shared structure and one-point calibration. MAE and PI widths are nats, pooled equally over capabilities. Family comparisons use registered dev splits: 17 pruning sources, six quantization states, 12 distillation runs. Specific forms: v53 power, v55 low-order 2D, v56 F2 with L_0 . Distillation D_0 is unavailable (inactive column). Gamma ranges describe within-source fits. Bootstrap parameter CIs are in the accompanying summary. K1 costs one compressed measurement per source; scores exclude that cell. PI entries give coverage/full width for K0 then K1, from per-capability absolute LOSO residual quantiles. Confirmation excludes three new sources; independent pair evaluation excludes all four pair sources from fitting and its 13-source interval bank. Quantization tests omit the target source from fitting; 2D K1 scales its predicted curve and sep K1 calibrates the separable amplitude.

D Complete Prediction Results

Tables 12–14 give every registered test in three lines (candidate, strongest same-input baseline, strongest simple baseline) with improvements and provenance codes; Table 15 the per-stage, per-protocol results of the Pythia-1B and 6.9B pairs; Tables 16 and 17 the confirmation panel and the grouped-quantization tests; Fig. 5 the paired improvements behind Fig. 3.

D.1 Pythia-1B and 6.9B pairs under two protocols

The second frozen round (Table 15) added seven pruning densities (0.9, 0.8, 0.75, 0.7, 0.65 and 0.6 as interpolation, 0.55 as extrapolation) and five bit-widths (8, 6, 5, 4, 3) on Pythia-1B at steps 32k and 112k. Two fitting protocols were frozen at the same time. Protocol A fits each candidate on the full nine-state development panel. Protocol B fits on development states at step 64k or below, so that its 112k rows test a training-stage extrapolation on top of the size change. The 5-bit prediction of the class-indicator candidate is a fixed rule, the average of its committed 4- and 6-bit predictions, chosen so that no 5-bit measurement enters any fit. Candidate names follow §3.2: power is the shared power form, A2 the per-density source regression with a fixed interpolation rule, A1 its $\gamma = 1$ variant, cont a continuous two-term polynomial in $1 - d$, and the three source-free baselines are strength-only, the median development curve, and zero change. For quantization, full uses $\{N_0, L_0, D_0\}$, no-D0 drops the training-stage term, and mean and median are per-bit constants from the development panel.

Per capability, at 112k under protocol A the source regression reaches MAE 0.111 (math), 0.091 (code) and 0.135 (QA); the strongest source-free alternative is strength-only for math (0.117) and the median curve for code (0.107) and QA (0.261). At 32k the QA error of the source regression rises to 0.339 while the median curve stays at 0.193, which accounts for most of the aggregate gap. Under protocol B at 112k the power and A2 forms are ahead of strength-only at $d = 0.55$ (0.96 and 1.00 against 1.33), but with errors near one nat the ordering carries little practical value. The pair cost 22 A100 minutes.

The 6.9B pair, a size extrapolation of about five times, was measured under the same freeze (36 A100 minutes, reference model on CPU). Its dense anchors are math 1.27, code 1.30 and QA 5.11 at 32k, and 1.11, 1.21 and 5.09 at 112k. Per capability under protocol A at 112k, the source regression reaches 0.23 (math), 0.19 (code) and 0.96 (QA) against 0.05, 0.07 and 0.15 for the median curve. Under protocol B the source forms lead on code only (A2 0.058 and 0.083 at the two stages against 0.208 and 0.074 for the median curve) and trail on QA (0.32 and 0.20 against 0.08 and 0.15). At $d = 0.55$ the source-free curves win under both protocols, with source-conditioned errors between 0.55 and 2.2 nats. For quantization under protocol A the class-indicator candidate is worst or near worst above 3 bits (0.25 and 0.27 against 0.03 and 0.04 for the per-bit median) and overshoots by about four nats at 3 bits. Protocol B compresses the differences above 3 bits to the 0.02 to 0.05 range and leaves the candidate ahead at 3 bits only for the late stage (1.29 against 3.8 to 4.1).

Test; unseen axis; [origin]	Line	math	code	QA
pruning: source transfer at fixed d : 3×3 Pythia panel, leave-one-step-out held-out pretraining stage (in-range) $[L \cdot F \cdot R \cdot A]$	cand.: config-indicator OLS	0.253 (+0.16)	0.292 (+0.22)*	0.806 (-0.31)
	same-input: no- D_0	0.613	0.693	0.774
	simple: per-config	0.413	0.516	0.500
	median; zero change			
quantization: source transfer at fixed b : 3×3 Pythia panel, leave-one-step-out held-out pretraining stage (in-range) $[L \cdot F \cdot R \cdot A]$	cand.: config-indicator OLS	1.144 (+0.27)	0.626 (+0.93)*	1.989 (-0.52)
	same-input: no- D_0	1.989	2.077	1.964
	simple: per-config	1.410	1.553	1.473
	median			
pruning: new stage 96k on 160M and 1.4B, all measured d stage 96k (interpolation between 64k and 143k) $[P \cdot F \cdot R \cdot A]$	cand.: config-indicator OLS	0.090 (+0.25)	0.128 (+0.05)	0.426 (+0.01)
	same-input: no- D_0	0.343	0.179	0.438
	simple: per-config	0.367	0.438	0.832
	median			
quantization: new stage 96k on 160M, 410M, 1.4B; $b \geq 4$ (near-zero regime) stage 96k (interpolation) $[P \cdot - \cdot R \cdot A]$	cand.: config-indicator OLS	0.029 (+0.06)	0.104 (-0.01)	0.173 (-0.04)
	same-input: none available	—	—	—
	simple: per-bit	0.084	0.094	0.134
	median			
quantization: new stage 96k on 160M, 410M, 1.4B; int3 (collapse regime) stage 96k (interpolation) $[P \cdot - \cdot R \cdot A]$	cand.: config-indicator OLS	1.488 (+0.45)	1.963 (-1.12)	2.046 (-0.32)
	same-input: none available	—	—	—
	simple: per-bit	1.934	0.843	1.727
	median			
distillation: δ_c source transfer: 3×3 LoRA panel, leave-one-step-out held-out pretraining stage (in-range) $[L \cdot F \cdot R \cdot A]$	cand.: linear $\{N_0, L_0, D_0\}$	0.014 (+0.01)	0.057 (-0.01)	0.263 (-0.10)
	same-input: no- D_0	0.026	0.044	0.265
	simple: constant (mean); constant (median)	0.031	0.061	0.159

Table 12: Source axis: does the pre-compression source state predict the response at a fixed intervention setting? MAE in nats/token per capability; bold = smallest in the group; the number in parentheses is the improvement of the candidate over the strongest listed baseline (positive = candidate better); * = bootstrap 95% interval of that improvement excludes zero (computed only for the leave-one-out panels). Origin code: prediction origin (P frozen prospective, L leave-one-out) · same-input baseline origin · simple baseline origin (F pre-specified with the candidate, R computed after the test, — none) · selection (A: all pre-specified forms reported). Where several same-input or simple baselines exist, the strongest per capability is shown and named; that choice is retrospective. Config-indicator OLS uses $\{N_0, L_0, D_0\}$ with one coefficient set per configuration. Generated by v52_prediction_tables.py.

Test; unseen axis; [origin]	Line	math	code	QA
pruning: unseen d 0.65 and 0.55, dev sources + 410M@96k densities 0.65 (interp.) and 0.55 (extrap.) pooled [P · R · F · A]	cand.: shared power form	0.371 (-0.09)	0.524 (-0.24)	0.759 (+0.06)
	same-input: A2 (per- d regr.)	0.283	0.279	0.818
	simple: strength-only	1.369	1.635	1.608
pruning: new source 1B@96k, $d = 0.65$ (interp.) in-range size; stage 96k [P · F · F · A]	cand.: shared power form	0.020 (-0.01)	0.092 (-0.06)	0.466 (-0.09)
	same-input: A1 ($\gamma=1$); A2 (per- d regr.)	0.009	0.031	0.380
	simple: median curve	0.147	0.076	0.569
pruning: new source 1B@96k, $d = 0.55$ (extrap.) in-range size; stage 96k [P · F · F · A]	cand.: shared power form	0.930 (-0.66)	1.550 (-1.53)	1.426 (-0.90)
	same-input: A2 (per- d regr.)	1.051	1.358	1.009
	simple: strength-only	0.273	0.018	0.530
pruning: new source 1B@32k+112k, d 0.9–0.6 (interp.) in-range size; stages 32k, 112k [P · F · F · A]	cand.: shared power form	0.215 (-0.05)	0.191 (-0.07)	0.261 (-0.03)
	same-input: A2 (per- d regr.)	0.205	0.119	0.237
	simple: median curve; strength-only	0.169	0.128	0.227
pruning: new source 1B@32k+112k, $d = 0.55$ (extrap.) in-range size; stages 32k, 112k [P · F · F · A]	cand.: shared power form	1.790 (-0.85)	1.488 (-1.40)	2.225 (-1.83)
	same-input: A1 ($\gamma=1$); A2 (per- d regr.)	1.896	1.386	1.744
	simple: strength-only	0.939	0.089	0.393
pruning: new source 6.9B@32k+112k, d 0.9–0.6 (interp.) size $5\times$ beyond dev range; stages 32k, 112k [P · F · F · A]	cand.: shared power form	0.299 (-0.18)	0.160 (-0.02)	0.909 (-0.79)
	same-input: A2 (per- d regr.)	0.292	0.144	0.982
	simple: median curve	0.120	0.144	0.120
pruning: new source 6.9B@32k+112k, $d = 0.55$ (extrap.) size $5\times$ beyond dev range; stages 32k, 112k [P · F · F · A]	cand.: shared power form	0.615 (+0.00)	0.547 (-0.01)	4.472 (-3.57)
	same-input: A1 ($\gamma=1$); A2 (per- d regr.)	0.618	0.535	2.850
	simple: median curve; strength-only	0.703	0.712	0.904

Table 13: Configuration axis, pruning: does a relation fit at seen densities and sources predict unseen densities, sizes and stages? Protocol A (full development panel) for the two P1-v2 pairs and the single-stage 1B@96k prospective; protocol B in Table 15. Format, bold, improvement in parentheses, and origin code as in Table 12.

Test; unseen axis; [origin]	Line	math	code	QA
quantization: new source 1B@96k, int4 in-range size; stage 96k; seen bit $[P \cdot F \cdot F \cdot A]$	cand.: config-indicator OLS	0.116 (-0.05)	0.046 (-0.03)	0.059 (-0.00)
	same-input: no- D_0	0.184	0.394	0.054
	simple: per-bit median	0.068	0.012	0.253
quantization: new source 1B@96k, int3 in-range size; stage 96k; seen bit $[P \cdot F \cdot F \cdot A]$	cand.: config-indicator OLS	0.690 (+0.72)	0.318 (-0.32)	0.182 (+0.26)
	same-input: no- D_0	2.324	3.423	1.635
	simple: per-bit median	1.414	0.001	0.446
quantization: new source 1B@32k+112k, $b \geq 4$ (incl. 5-bit rule) in-range size; stages 32k, 112k $[P \cdot F \cdot F \cdot A]$	cand.: config-indicator OLS	0.064 (-0.04)	0.041 (-0.01)	0.157 (-0.10)
	same-input: no- D_0	0.118	0.156	0.153
	simple: per-bit median	0.020	0.028	0.052
quantization: new source 1B@32k+112k, int3 in-range size; stages 32k, 112k $[P \cdot F \cdot F \cdot A]$	cand.: config-indicator OLS	0.256 (+1.21)	0.362 (+1.18)	1.972 (-0.97)
	same-input: no- D_0	3.218	3.565	2.897
	simple: per-bit mean; per-bit median	1.466	1.537	1.000
quantization: new source 6.9B@32k+112k, $b \geq 4$ (incl. 5-bit rule) size $5 \times$ beyond dev range; stages 32k, 112k $[P \cdot F \cdot F \cdot A]$	cand.: config-indicator OLS	0.181 (-0.14)	0.177 (-0.13)	0.428 (-0.39)
	same-input: no- D_0	0.169	0.154	0.426
	simple: per-bit median; zero change	0.036	0.043	0.034
quantization: new source 6.9B@32k+112k, int3 size $5 \times$ beyond dev range; stages 32k, 112k $[P \cdot F \cdot F \cdot A]$	cand.: config-indicator OLS	3.233 (-1.19)	3.229 (-1.42)	5.306 (-3.56)
	same-input: no- D_0	4.199	3.176	5.605
	simple: per-bit mean	2.046	1.805	1.744
distillation: unseen pool U225 from endpoint pools U75/U600 (Gemma-3-1B) pool (reuse count in-range; sampling protocol differs) $[P \cdot F \cdot F \cdot A]$	cand.: E -only (reuse count)	0.191 (-0.07)	0.117 (-0.05)	0.706 (+0.00)
	same-input: $T+E$ joint	0.215	0.145	0.710
	simple: constant	0.122	0.069	1.449

Table 14: Configuration axis, quantization and distillation: unseen sizes and stages at seen bit-widths (protocol A; the 5-bit prediction is a fixed interpolation rule), and an unseen distillation pool. Format, bold, improvement in parentheses, and origin code as in Table 12.

Source	Prot.	Regime	power	A2	A1	cont	str.-only	median	zero	best
1b@32000	A	prune interp d0.9-0.6	0.254	0.261	0.393	0.275	0.363	0.204	0.412	median-curve
1b@32000	A	prune extrap d0.55	2.272	2.130	2.394	2.221	0.637	1.509	2.434	strength-only
1b@32000	B	prune interp d0.9-0.6	0.249	0.218	0.321	0.239	0.236	0.224	0.412	A2
1b@32000	B	prune extrap d0.55	1.772	1.777	2.151	1.863	1.531	1.645	2.434	strength-only
1b@112000	A	prune interp d0.9-0.6	0.191	0.112	0.300	0.175	0.365	0.173	0.400	A2
1b@112000	A	prune extrap d0.55	1.398	1.235	1.838	1.356	0.311	1.311	2.237	strength-only
1b@112000	B	prune interp d0.9-0.6	0.223	0.177	0.355	0.235	0.219	0.195	0.400	A2
1b@112000	B	prune extrap d0.55	0.957	0.995	1.685	1.127	1.333	1.447	2.237	power
6.9b@32000	A	prune interp d0.9-0.6	0.476	0.482	0.608	0.501	0.483	0.165	0.160	zero
6.9b@32000	A	prune extrap d0.55	1.635	1.423	0.939	1.457	1.493	0.275	0.889	median-curve
6.9b@32000	B	prune interp d0.9-0.6	0.160	0.166	0.249	0.187	0.163	0.136	0.160	median-curve
6.9b@32000	B	prune extrap d0.55	0.705	0.549	0.677	0.630	0.252	0.148	0.889	median-curve
6.9b@112000	A	prune interp d0.9-0.6	0.436	0.463	0.610	0.479	0.376	0.092	0.294	median-curve
6.9b@112000	A	prune extrap d0.55	2.121	1.939	2.224	2.061	0.191	1.272	2.198	strength-only
6.9b@112000	B	prune interp d0.9-0.6	0.131	0.106	0.235	0.140	0.111	0.095	0.294	median-curve
6.9b@112000	B	prune extrap d0.55	1.441	1.387	1.892	1.525	1.294	1.408	2.198	strength-only
Source	Prot.	Regime	full	no-D0	mean	median	zero	best		
1b@32000	A	quant ≥ 4 -bit	0.103	0.174	0.197	0.031	0.046	per-bit-median		
1b@32000	A	quant int3	1.050	2.648	2.903	1.438	3.108	full		
1b@32000	A	quant int5 (rule)	0.179	0.272	0.254	0.044	0.017	zero		
1b@32000	B	quant ≥ 4 -bit	0.028	0.032	0.027	0.026	0.046	per-bit-median		
1b@32000	B	quant int3	0.606	1.004	0.627	0.803	3.108	full		
1b@32000	B	quant int5 (rule)	0.025	0.013	0.035	0.034	0.017	noD0		
1b@112000	A	quant ≥ 4 -bit	0.071	0.111	0.183	0.036	0.063	per-bit-median		
1b@112000	A	quant int3	0.676	3.805	0.802	1.231	5.777	full		
1b@112000	A	quant int5 (rule)	0.151	0.138	0.251	0.035	0.025	zero		
1b@112000	B	quant ≥ 4 -bit	0.038	0.042	0.038	0.039	0.063	per-bit-mean		
1b@112000	B	quant int3	1.120	1.981	3.106	3.472	5.777	full		
1b@112000	B	quant int5 (rule)	0.046	0.019	0.027	0.025	0.025	noD0		
6.9b@32000	A	quant ≥ 4 -bit	0.253	0.317	0.190	0.034	0.033	zero		
6.9b@32000	A	quant int3	3.973	5.103	3.300	1.835	2.711	per-bit-median		
6.9b@32000	A	quant int5 (rule)	0.373	0.457	0.237	0.067	0.029	zero		
6.9b@32000	B	quant ≥ 4 -bit	0.033	0.050	0.029	0.025	0.033	per-bit-median		
6.9b@32000	B	quant int3	0.489	1.792	0.174	0.407	2.711	per-bit-mean		
6.9b@32000	B	quant int5 (rule)	0.046	0.052	0.058	0.057	0.029	zero		
6.9b@112000	A	quant ≥ 4 -bit	0.271	0.183	0.155	0.042	0.061	per-bit-median		
6.9b@112000	A	quant int3	3.873	3.550	0.430	1.896	6.442	per-bit-mean		
6.9b@112000	A	quant int5 (rule)	0.402	0.247	0.253	0.033	0.027	zero		
6.9b@112000	B	quant ≥ 4 -bit	0.048	0.060	0.042	0.045	0.061	per-bit-mean		
6.9b@112000	B	quant int3	1.294	4.033	3.770	4.137	6.442	full		
6.9b@112000	B	quant int5 (rule)	0.043	0.025	0.025	0.023	0.027	per-bit-median		

Table 15: P1-v2 new-source frozen prospectives (MAE over three capabilities, nats/token). Protocol A fits on the full nine-state development panel; protocol B on step $\leq 64k$ only, so its 112k rows are training-stage extrapolations. 1B sources are in-range sizes and 6.9B is a $\sim 5\times$ size extrapolation. Predictions were committed before measurement; per-capability and per-point values are in the results ledger.

Candidate	P	test MAE			signed bias (pred. - obs.)		
		math	code	QA	math	code	QA
power (selected)	5	0.243	0.236	0.678	+0.216	+0.236	+0.382
A2	20	0.230	0.222	0.682	+0.222	+0.212	+0.293
A1	4	0.363	0.403	0.738	+0.210	+0.206	+0.304
cont	8	0.313	0.305	0.746	+0.116	+0.107	+0.249
strength only	2	0.450	0.488	0.579	+0.443	+0.486	+0.579
median curve	0	0.277	0.214	0.221	+0.277	+0.214	+0.186
zero	0	0.522	0.530	0.185	-0.522	-0.530	+0.185

Table 16: Round-3 pruning deliverable. Forms: power $(\beta_c \cdot \phi)((1-d)/0.3)^{\gamma_c}$; A2 = per-density OLS on ϕ at five anchors with linear interpolation; A1 = power form with $\gamma_c=1$; cont = $(\beta_c \cdot \phi)s + (\zeta_c \cdot \phi)s^2$, $s=1-d$; strength-only $A((1-d)/0.3)^{\gamma}$, the median development curve, and zero change are source-free. Inputs $\phi = [1, \log \tilde{N}_0, \tilde{L}_{0,c}, \log \tilde{D}_0]$ (standardized on the development set); development set = 17 Pythia states (160M–6.9B, steps 16k–143k, densities ≥ 0.55); the selected form was chosen by a pre-committed leave-one-source-out rule (tie within 0.02 nats resolved toward the continuous, zero-at- $d=1$ form with fewer parameters). Test = three new checkpoints (410M@48k, 1.4B@112k, 6.9B@80k) at $d \in \{0.85, 0.675, 0.575\}$, predictions committed before measurement; MAE and mean signed error (prediction minus observation) over nine cells per capability. Coefficients in Appendix Table 5. Gain of the selected form vs A2 (same input): math -0.013, code -0.014, qa +0.004; vs the strongest source-free curve: math +0.034, code -0.023, qa -0.494. Generated by v57_round3_tables.py.

Candidate	P	bit test (12)			granularity test (18)			joint, 1B@96k (3)		
		math	code	QA	math	code	QA	math	code	QA
low-order 2D	20	0.193	0.214	0.436	0.219	0.319	0.986	0.345	0.097	0.269
separable	6	0.767	0.798	1.427	0.581	0.643	1.612	0.586	0.380	0.679
same input interpolation	16	1.296	1.381	1.942	0.606	0.711	1.497	0.538	0.241	0.392
median	0	0.553	0.726	0.537	1.123	1.222	1.220	0.152	0.270	0.302
mean	0	1.416	1.556	1.495	1.857	1.906	2.019	1.491	1.558	1.813
zero	0	0.450	0.532	0.480	1.239	1.346	1.239	0.300	0.339	0.104

Table 17: Round-3 grouped-quantization deliverable (MAE, nats). Forms: low-order 2D = $\phi \cdot [1, u, v, uv, u^2]$ with $u = \log_2 q_{\max}$ (centred) and $v = \log_2(g/128)$, one 4-vector of state coefficients per term; separable = $(\beta_c \cdot \phi) q_{\max}^{-p_c} (g/128)^{q_c}$; same-input interpolation = bilinear between the four development configurations; median and mean = per-configuration statistics interpolated the same way (source-free). Same inputs ϕ as Table 16; development = six Pythia states (160M/410M/1.4B at steps 16k and 143k) $\times b \in \{3, 5\} \times g \in \{64, 256\}$ under symmetric round-to-nearest with contiguous input groups of g weights; all predictions committed before the tests. Bit test: unseen $b=4$ at $g \in \{64, 256\}$ on the dev states; granularity test: unseen $g=128$ at $b \in \{3, 4, 5\}$ on the dev states; joint test: new state 1B@96k at $g=128$. All candidates and baselines are reported on every test (pre-committed rule; no selection by outcome). Signed bias of the 2D form (math/code/QA): bit +0.108/+0.060/+0.077; granularity +0.162/+0.137/+0.129; joint +0.345/+0.097/+0.250. Generated by v57_round3_tables.py.

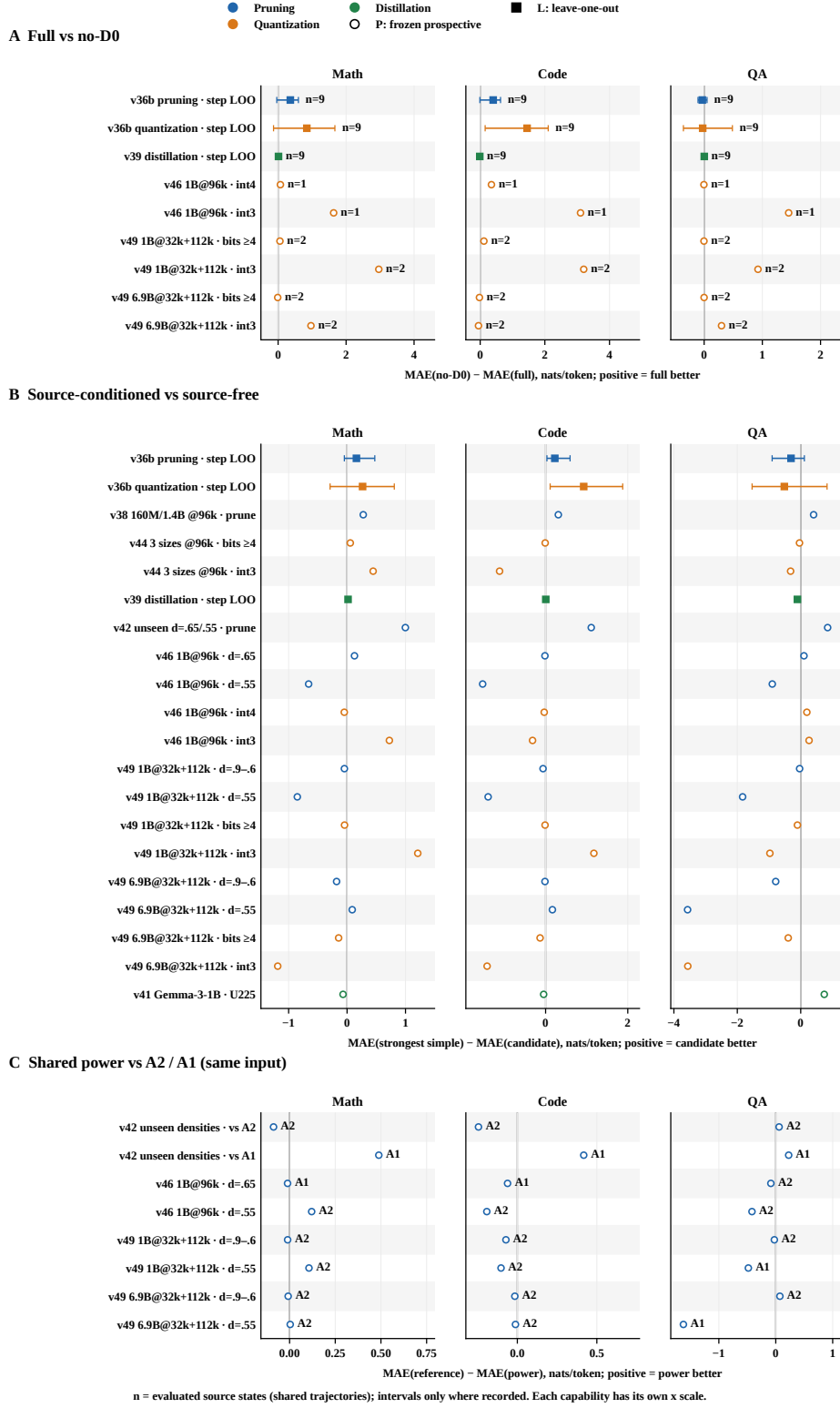


Figure 5: Paired improvements, zero line marked, positive means the first-named model is better. A: full model against its no- D_0 variant, per arm and capability; intervals only where a cluster bootstrap was computed (leave-one-stage-out panels), n = evaluated source states. B: source-conditioned candidate against the strongest source-free baseline for all twenty registered tests (hollow = frozen prospective, filled = leave-one-out). C: shared power form against the same-input alternatives A1 and A2 on the pruning configuration tests. Generated by plot_fig2_gains.py from `27` v52 groups and the v36b/v39/v42/v46/v49 result files.

E Capability-Loss Measurement Checks

Capability specificity as a prediction test. On a six-model, five-density pruning panel (d from 0.9 to 0.7), the response on the primary benchmark of each capability was used to predict the response on the secondary benchmark of the same capability through a two-parameter affine map fit out of fold, and compared with the best cross-capability predictor chosen the same way. With whole models held out, the same-capability gain is +0.04 nats for math (95% model-bootstrap interval $[0.03, 0.05]$) and +0.14 for QA ($[0.02, 0.27]$); with whole densities held out it is +0.14 ($[0.03, 0.30]$) and +0.21 ($[0.02, 0.52]$). For code the cross-capability predictor is as good or better ($-0.06, [-0.16, -0.00]$ with densities held out), so code specificity is not established at this resolution (Fig. 6). Six model clusters give descriptive intervals only.

Loss and behavior. Loss-to-accuracy links were fit on GSM8K accuracy, MBPP pass@1, and TriviaQA exact match and token F1 across seven pruned models; all four outcomes are compatible with one cross-family sigmoid in the loss, with no resolved family-specific term. A lower QA loss does not by itself imply better QA behavior: the four pruned OLMo-3-32B cells whose QA loss lies below dense show no increase in exact match. This audit covers pruning only.

Reference conditions. For a subset of models the direct-answer and given-reasoning-answer conditional distributions were measured in addition to the full-solution reference; they differ from it for math and QA and coincide for code, where the reference is the function body, so the numbers in this paper are specific to the full-solution reference and are not evidence of format robustness.

The loss is the conditional cross-entropy of the reference completion given the prompt, summed over completion tokens and divided by the number of completion tokens, as in Eq. equation 1. Prompts and references are tokenized with the model’s own tokenizer and truncated to 1024 tokens; prompt tokens are masked out of the loss. All within-model responses ΔL_c are differences in native-token nats on identical token sequences. Because native-token units differ across tokenizers, cross-family comparisons of absolute levels use per-byte normalization or are reported separately, and no cross-tokenizer quantity is treated as a common numerical scale. Negative responses are retained as measured; nothing is clipped at zero.

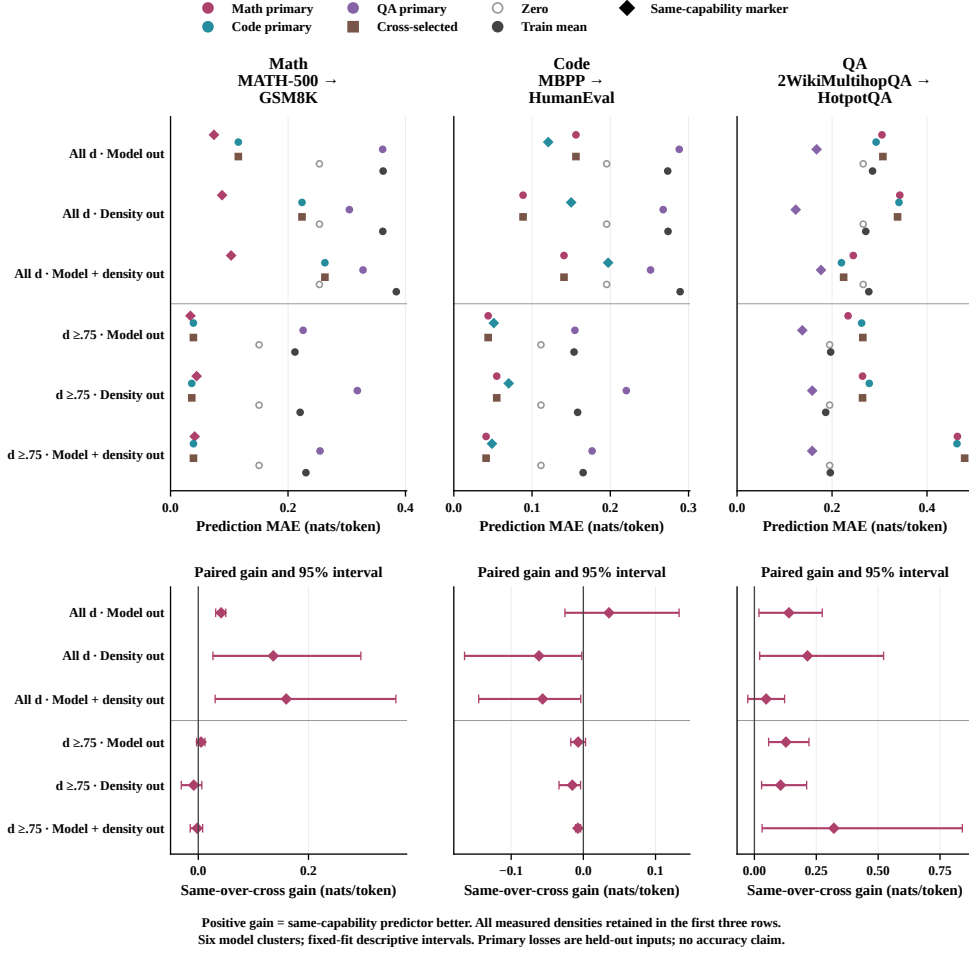


Figure 6: Measurement support for capability specificity. Top: out-of-fold MAE of predicting the secondary-benchmark response of each capability from the primary-benchmark response of the same capability (diamond) or of another capability, against zero and train-mean controls, on the six-model, five-density pruning panel; rows are hold-out protocols with all densities and with $d \geq 0.75$ only. Bottom: paired same-over-cross gain with 95% model-bootstrap intervals (six clusters, fixed out-of-fold fits). Generated by plot_fig4_measurement.py from v26-loss-validity-pred/summary.json.

F Heterogeneous-Model Results

Pruning. Table 18 gives per-capability damage at two densities, the density at which math damage first exceeds one nat, and how QA ranks before that cliff. Magnitudes differ by two orders across series at the same density (OLMo-3-32B +0.01 against Gemma-3-27B +4.0 on math at $d = 0.7$). The ordering of capabilities is series-specific: in the Qwen3 sizes and in OLMo-3-32B, QA is the least-damaged capability at every pre-cliff density and improves, by up to -0.79 nats at Qwen3-4B; in every Gemma-3 size, in Gemma-4-31B, and in Muse-30B, QA is the most-damaged capability at the mildest densities (Gemma-4-31B: +2.89 QA against +0.68 math at $d = 0.7$); OLMo-3-7B lies in between with differences of at most 0.15 nats. A noise screen sets the scale: the median $|\Delta L|$ at $d = 0.9$ is 0.013 nats, and 29 of the 62 nominal improvement cells across all arms exceed twice that median; the rest are treated as zero. Cliffs occur between $d = 0.5$ and 0.7 for Qwen3 and Gemma, at 0.55 for Muse-30B and 0.35 for OLMo-3-7B, and had not occurred by $d = 0.3$ for OLMo-3-32B. Robustness is

Model	ΔL_c at $d = 0.7$			math@0.5	d_{math}^*	QA least worst	min QA (d)
	math	code	QA				
Qwen3-0.6B	+0.49	+0.56	-0.26	+4.72	0.60	3/3 0/3	-0.26 (0.70)
Qwen3-1.7B	+0.24	+0.19	-0.51	+3.60	0.60	6/6 0/6	-0.90 (0.62)
Qwen3-4B	-0.00	-0.09	-0.79	+1.31	0.50	5/5 0/5	-1.96 (0.55)
Qwen3-8B [†]	+0.03	+0.10	-0.41	–	none by 0.60	4/4 0/4	-1.02 (0.60)
Qwen3-14B [†]	+0.06	+0.03	-0.45	–	none by 0.60	3/4 0/4	-0.80 (0.60)
Gemma-3-270M	+3.49	+4.04	+2.54	+11.21	0.70	0/2 1/2	+0.21 (0.90)
Gemma-3-1B	+1.31	+1.23	+0.83	+13.78	0.70	0/5 4/5	+0.19 (0.90)
Gemma-3-4B	+0.83	+1.47	+0.24	+7.00	0.60	1/3 2/3	+0.05 (0.90)
Gemma-3-12B	+0.68	+0.46	+0.54	+17.10	0.60	0/3 2/3	+0.07 (0.90)
Gemma-3-27B	+4.04	+4.69	+3.59	+15.21	0.70	1/2 1/2	+0.01 (0.80)
Gemma-4-31B	+0.68	+0.77	+2.89	+16.06	0.60	0/3 3/3	+0.22 (0.90)
Muse-30B	+0.17	+0.14	+0.20	+3.53	0.55	1/4 3/4	+0.02 (0.90)
OLMo-3-7B	+0.03	+0.03	+0.14	+0.22	0.35	4/8 3/8	-0.02 (0.55)
OLMo-3-32B	+0.01	+0.03	+0.02	+0.09	none by 0.30	7/10 0/10	-0.24 (0.30)

Table 18: Per-capability pruning damage on the heterogeneous panel (nats/token, native tokenizer units; within-model differences). d_{math}^* is the largest measured density with $\Delta L_{\text{math}} \geq 1$; models are grouped by series (Qwen3, Gemma-3, Gemma-4, Muse, OLMo-3); “math@0.5” is ΔL_{math} at $d = 0.5$; “QA least | worst” counts, over the pre-cliff densities, how often QA is the least-damaged and how often the most-damaged capability the last column is the most negative QA response and the density at which it occurs. [†]prospective additions measured at four densities only. Generated by v51_panel_tables.py from prune_losses.json.

Model	Series	ΔL_c at int4			ΔL_c at int3			bits
		math	code	QA	math	code	QA	
Qwen3-0.6B	Qwen3	+0.56	+0.73	+0.21	+8.83	+10.83	+6.61	8,6,5,4,3
Qwen3-1.7B	Qwen3	+0.40	+0.16	-0.21	+9.53	+9.46	+8.10	8,6,5,4,3
Qwen3-4B	Qwen3	+0.13	+0.19	+0.39	+10.73	+11.85	+5.99	8,6,5,4,3
Qwen3-8B [†]	Qwen3	+0.28	+0.48	-0.80	+12.86	+13.55	+9.45	8,6,5,4,3
Qwen3-14B [†]	Qwen3	+0.09	+0.31	-0.23	+9.30	+10.79	+6.45	8,6,4,3
Gemma-3-270M	Gemma-3	+1.99	+2.35	+0.88	+26.15	+27.94	+25.93	8,6,5,4,3
Gemma-3-1B	Gemma-3	+0.95	+0.59	+0.88	+15.33	+14.12	+9.80	8,6,5,4,3
Gemma-3-4B	Gemma-3	+0.72	+0.50	-0.27	+21.73	+19.24	+15.30	8,6,5,4,3
Gemma-3-12B	Gemma-3	+0.40	+0.32	-0.42	+20.19	+22.92	+20.75	8,6,5,4,3
Gemma-3-27B	Gemma-3	+0.33	+0.23	-0.67	+18.46	+17.15	+16.10	8,6,5,4,3
Gemma-4-31B	Gemma-4	+0.94	+0.58	+1.28	+20.35	+19.85	+14.90	8,6,5,4,3
Muse-30B	Muse	+1.58	+2.06	-0.11	+11.14	+11.18	+8.36	8,6,5,4,3
OLMo-3-7B	OLMo-3	+0.16	+0.21	-0.07	+4.70	+4.63	+2.76	8,6,5,4,3
OLMo-3-32B	OLMo-3	+0.04	+0.06	+0.02	+0.68	+1.22	+0.38	8,6,5,4,3

Table 19: Per-capability quantization damage (per-output-channel symmetric RTN, weights only; nats/token). int8 and int6 are within 0.01 nats of dense for every listed model and are omitted. [†]prospective additions. Generated by v51_panel_tables.py from quant_losses.json.

not monotone in size within a series (Gemma-3-12B is the most robust Gemma-3 size; 27B collapses earlier). We read this as heterogeneity a predictor has to reproduce, not as an invariant.

Quantization. Table 19 covers 14 models. Int8 and int6 stay within 0.01 nats of dense. Int4 damages small models (Gemma-3-270M +2.0, +2.4, +0.9) and barely touches large ones (Gemma-3-27B +0.33, +0.23, -0.67); a QA improvement under int4 recurs in five models and clears the noise screen. Int3 collapses 12 of the 14 models by 4.6 to 28 nats; OLMo-3-7B loses 2.8 to 4.7 and OLMo-3-32B only 0.4 to 1.2, so the collapse is common but not universal, and OLMo-3 is the robust series under both interventions. Within the tested bit-widths a learned exponential $q_c e^{-kb}$ fits held-out bits better than the fixed $q_c 4^{-b}$ shape

(pooled MAE 6.7 against 7.6 nats), which says the fixed base is wrong for this quantizer and nothing more; the squared step-size ratio between 4 and 5 bits is $(7/15)^2 \approx 0.22$ (Appendix A), and a loss ratio would follow only under error-distribution assumptions we do not test.

Distillation. Black-box traces from two API teachers (600 per domain) train Gemma-3 students under a full-trace and an answer-only recipe in a paired design. The answer-only recipe hurts math more (+0.19 to +0.22 against +0.09 to +0.12 nats relative to the initial student) while QA improves under every recipe (−0.58 to −1.44); because the recipes differ in supervised tokens and update counts, this pairing does not isolate reasoning coverage, and the controlled comparisons are those of §3.4.

What the endpoint does and does not say. The same heterogeneity appears on independent secondary benchmarks for math and QA, where the primary-benchmark response predicts the secondary one better than any other capability’s response does, with intervals excluding zero in most hold-out protocols; for code the cross-capability predictor is as good or better (Appendix E, Fig. 6). A lower QA loss does not by itself mean better QA behavior: in the four pruned OLMo-3-32B cells whose QA loss lies below dense, exact-match accuracy does not rise. “QA improves” in this paper refers to the loss.

F.1 Scale and family structure

Within-family robustness is non-monotonic in two families independently. Gemma-3 robustness peaks at 12B (27B collapses far earlier, +4.0 vs. +0.68 at $d = 0.7$), mirroring the prior ability-space observation of a Qwen3 14B peak. A mid-scale robustness peak is a reproducible pattern of scaling-down.

A candidate predictor, honestly retired. The top-concentration statistic κ_c tracked robustness within the Qwen3 ladder and was pre-registered as a predictor; the full grid retires it (cross-family inversion: OLMo, the most robust family, has the lowest concentration; within-Gemma top-end violation).

Matched-size panel (~ 30 B). At matched scale the family coefficient dominates: ΔL_{math} at $d = 0.7$ spans +0.02 (OLMo-3-32B), +0.17 (Muse-30B), +0.68 (Gemma4-31B), +4.04 (Gemma3-27B). OLMo-3-32B survives 70% pruning with $\leq +0.63$ nats on each capability. Family identity is a first-order determinant of prunability on this panel; which training-data or recipe properties carry it was not measured.

G Mechanistic Analyses

G.1 Capabilities occupy distinct regions of gradient space

Convergent validity (block structure). For each of nine benchmarks we measure a gradient signature (the benchmark-conditional diagonal Fisher) and compare all pairs. After removing the component shared across benchmarks, signatures of same-capability benchmarks are positively aligned (cosine +0.36 to +0.45) while signatures of different capabilities are anti-aligned (−0.23 to −0.25), in each of the ten models from five series for which the block analysis was run (Figs. 7–16); the C4 control sits between blocks. (preliminary evidence: point estimates; example-bootstrap, permutation, and prompt-stability tests are left to future work) The mathematics benchmarks cluster regardless of format, and within-block fine structure matches intuition (SVAMP, grade-school word problems, is the loosest member of the math block). A projected-gradient SVD version gives the same verdict in subspace terms: principal-angle similarity within capability exceeds cross-capability similarity by 2–4 $\times$.

Selective ablation (preliminary evidence: single model; random, magnitude-matched, and global-Fisher controls are left to future work). A first targeted test: select the top 0.05% of weight coordinates by each capability’s residual Fisher signature (excluding coordinates

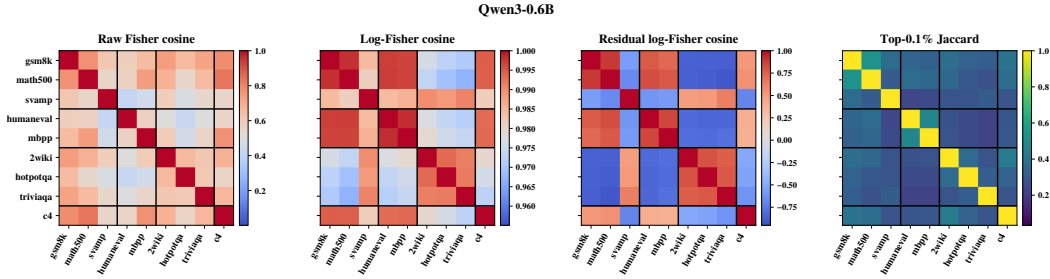


Figure 7: Capability regions in gradient space for Qwen3-0.6B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

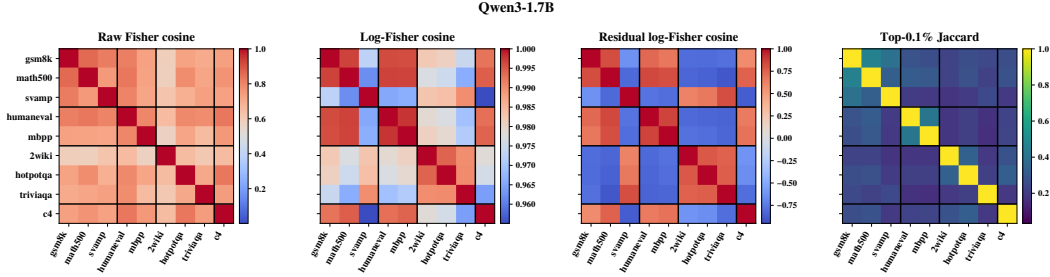


Figure 8: Capability regions in gradient space for Qwen3-1.7B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

shared with other capabilities) and zero them. The damage matrix is nearly diagonal (Gemma-3-1B; rows ablate, columns measure, nats/token):

Ablated region	math	code	QA
math	+4.63	+0.43	+0.05
code	+0.42	+7.14	+0.00
QA	+0.01	+0.00	+2.29

Zeroing 0.05% of weights, selected from gradients alone, selectively removes one capability while leaving the others nearly untouched (the QA ablation moves code by +0.0001). Pending multi-model replication and the control ablations above, we describe the regions as causally implicated and partially separable; we do not call them proven causal carriers; if confirmed, ablation itself is a form of targeted scaling-down.

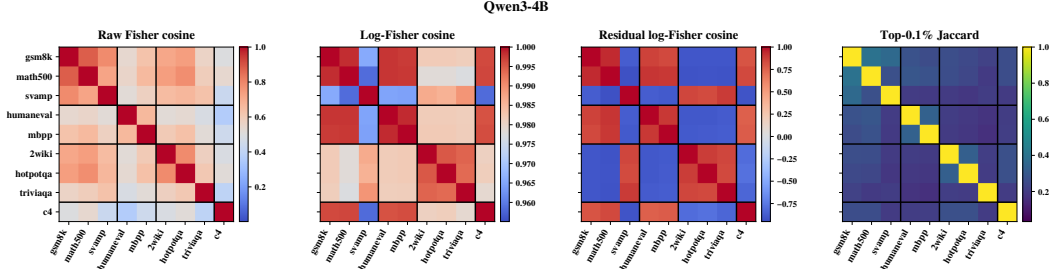


Figure 9: Capability regions in gradient space for Qwen3-4B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

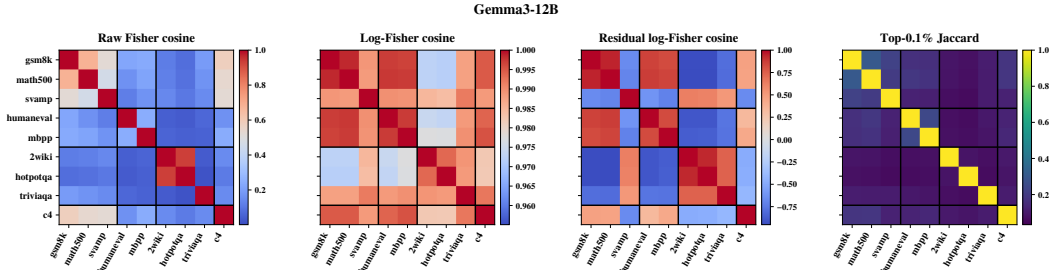


Figure 10: Capability regions in gradient space for Gemma3-12B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

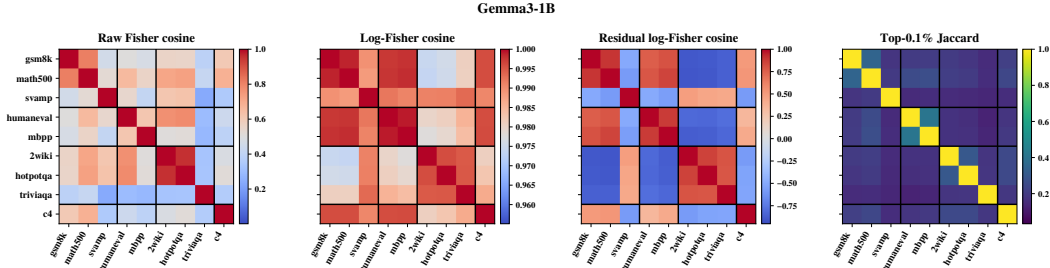


Figure 11: Capability regions in gradient space for Gemma3-1B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

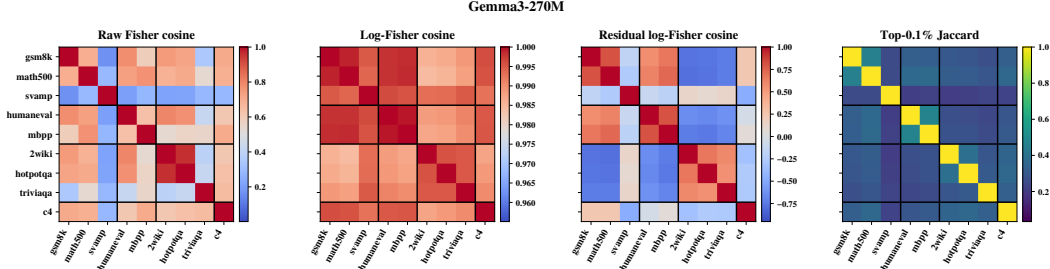


Figure 12: Capability regions in gradient space for Gemma3-270M: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

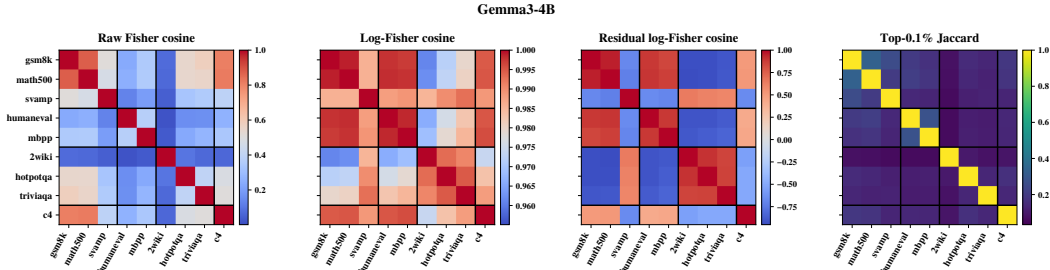


Figure 13: Capability regions in gradient space for Gemma3-4B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

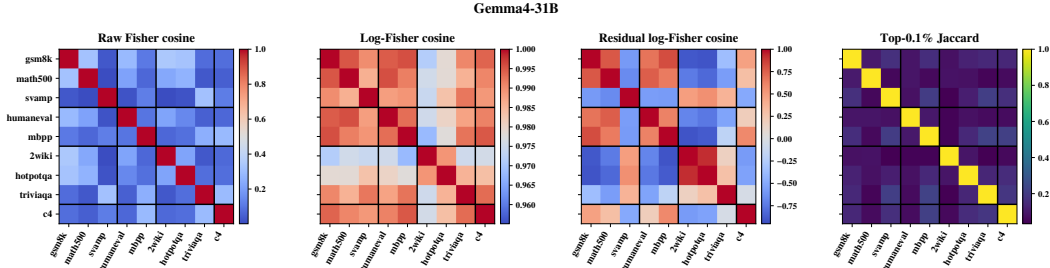


Figure 14: Capability regions in gradient space for Gemma4-31B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

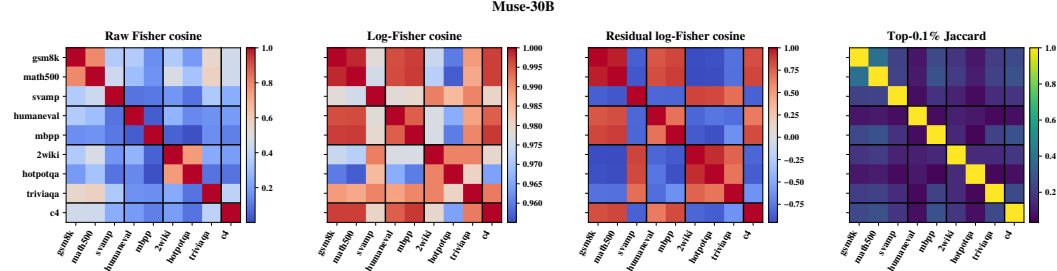


Figure 15: Capability regions in gradient space for Muse-30B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

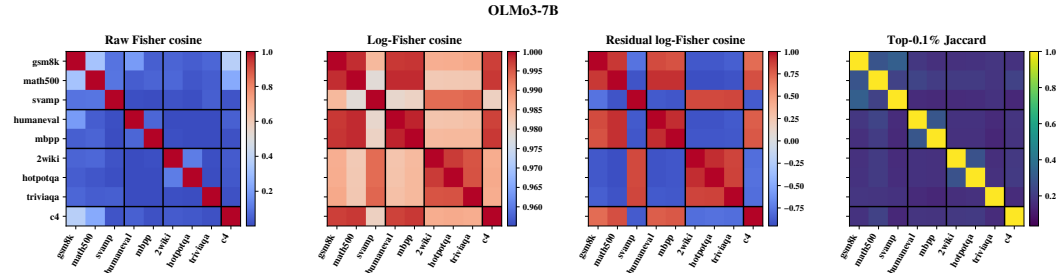


Figure 16: Capability regions in gradient space for OLMo3-7B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

G.2 The pruning law

The mechanism form below fits the entire pre-cliff grid. With the measured alignment term subtracted, the curvature residual follows a power law in the deleted capability mass $m_c(d)$:

$$\Delta L_c(d) = \underbrace{\bar{g}_c^\top \delta w_d}_{\text{measured}} + B \left[\underbrace{m_c(d)}_{\text{measured}} \right]^\gamma, \quad (7)$$

with in-sample $R^2 = 0.82\text{--}0.996$ over 24 capability $\times$ model fits. The held-out battery (1000-resample cell bootstrap) is where the form is tested: in leave-one-family-out transfer the shared- γ mechanism form attains MAE 2.17 nats versus 10.3 for per-capability raw sparsity and 8.2 for the direct-damage baseline (both CIs exclude zero), so the deleted-mass coordinate is what transfers across families; it also beats the direct-damage baseline in fit-shallow/predict-deep (3.11 vs. 4.24) and raw sparsity in leave-largest-model-out (1.41 vs. 2.15), while its edge over raw sparsity within-model does not reach significance (without the first-order term, QA is unfittable, $R^2 \leq 0.67$; with it, QA joins the law). Two structural results: (i) in the deleted-mass coordinate the exponent concentrates at $\gamma \in [1.2, 2.2]$ across all families, versus 3.2–8.7 in the raw $(1-d)$ coordinate, which is what makes the form portable; and (ii) within a model, γ is shared across capabilities (e.g. Qwen3-0.6B: 1.41/1.56/1.43; Gemma3-4B: 1.66/1.71): capability differences appear to be carried by the two measured quantities, and the fitted structure is two parameters per model. This mechanism form uses gradient information of the dense model and is therefore not a K0 predictor; it is reported as an upper comparator for the reduced forms of the main text, not as a bound.

G.3 The signed first-order term predicts, in both directions

The alignment term $\bar{g}_c^\top \delta w$ is the family coefficient that carries sign. Its sign predicts the presence or absence of the capability-selective improvement in each case measured: the QA term of Qwen3 is negative and QA improves under pruning (predicted -0.83 vs. measured -0.79 at 4B, $d = 0.7$); the QA term of Gemma is positive and no improvement exists at any size; the term for OLMo-3-7B is positive (mild degradation, prediction $+0.015$ = measurement at $d = 0.8$, code) while the term for OLMo-3-32B is negative, and deep-pruning QA improvement duly appears. At the level of individual model $\times$ capability $\times$ density cells in the mild regime, sign prediction accuracy is 67.6% (Wilson 95% CI $[58.3, 75.7]$, $n=108$), above chance, with the family-level pattern (Qwen and OLMo-32B QA improvements, Gemma none, an OLMo size flip) called in advance. Beyond the mild regime, first- plus second-order predictions fail by orders of magnitude at the cliff in each family: the collapse is non-perturbative in loss space, confirming the two-regime structure of equation 7.

G.4 The cliff is a geometric threshold crossing

Measuring the capability geometry of the pruned models accounts for the location of the cliff on the three models tested. The capability-region mass retention (the fraction of the capability-exclusive top coordinates of the dense model still prominent after pruning) declines smoothly from the first densities (no cliff in geometry) while loss is still flat; the behavioral cliff occurs where retention crosses a critical value. The critical value is family-specific (Gemma: 0.035–0.10 across 1B/4B and all capabilities; Qwen: 0.13–0.23) but consistent within a family across sizes and capabilities: the cliff boundary $d_c^*(N_0, f)$ reduces to one critical-retention constant per family, turning the least elegant fitted parameter of the two-regime law into a measurable, smoothly evolving order parameter. (preliminary evidence: three models, two families; a multi-model extension is left to future work)

H Recovery and Cross-Method Analyses

H.1 Recovery arm: redeemable damage and the fitted plateau

Recovery training on general text fits the recovery law cleanly (Gemma3-270M pruned to $d = 0.6$; $R^2 \approx 1.0$): $\beta'_{\text{math}} = 1.23$, $\beta'_{\text{code}} = 1.02$, with fitted plateaus under the fixed LoRA recipe and the largest measured budget of 16M tokens, $L_c^{\text{plateau}} - L_c^{\text{dense}}$, of 1.21 (math) and

1.93 (code) nats; about 80% of this pruning damage is redeemed within 8M tokens and the remainder persists up to the largest budget measured under this recipe, while QA recovers completely. The plateau is a fitted quantity at 16M tokens, not an established asymptote, and the saturating form does not describe the non-monotone aligned-data trajectory below. (preliminary evidence: one model, one density, one recipe; with three-point ladders the largest-budget extrapolation of the saturating form beats a carry-forward baseline in only 4 of 9 fits, so the ladder is too sparse to claim extrapolation, and a denser multi-model grid is left to future work) Two mechanism results: recovery on aligned data (teacher traces) heals substantially faster at small budgets (math 2.76 vs. 3.47 at 0.5M tokens), as the gradient-overlap prediction requires, but over-training on narrow aligned data re-damages the model (QA 4.95 $\rightarrow$ 9.75 from 0.5M to 8M tokens), while general-text recovery is monotone. Aligned recovery is fast at small budgets and harmful when over-trained on narrow data; general recovery is slow and monotone on this model. Quantization damage past the bit cliff is markedly less redeemable under the same recipe (residuals $\approx$ 2.6–2.8 after int3) than pruning damage at matched severity.

H.2 Resource-ratio models and method selection

Table 20 gives the per-policy results behind Fig. 4: mean regret against the measured oracle, agreement with the oracle’s method, the share of no-clear-winner cells, and the share of cells in which a fixed-method policy cannot meet the budget (it then falls back to the dense source, which is why its regret can be negative). Generated by v60_selection_maps.py.

Table 20: Leave-one-state-out compression selection. Regret is in nats; agreement, no-winner and over-budget shares are percentages.

Capability	Policy	Regret	Agree (%)	No winner (%)	Over budget (%)
Math	MAP	0.1936	95.8	46.4	0.0
Math	ORACLE	0.0000	100.0	46.4	0.0
Math	prune-only	0.0605	0.0	46.4	44.3
Math	quant-only	0.2089	96.5	46.4	0.0
Math	distill-only	-0.1585	2.1	46.4	63.0
Math	CHEAPEST	4.5839	79.9	46.4	0.0
Code	MAP	0.1743	94.8	56.7	0.0
Code	ORACLE	0.0000	100.0	56.7	0.0
Code	prune-only	0.0459	2.8	56.7	44.3
Code	quant-only	0.1943	94.1	56.7	0.0
Code	distill-only	-0.1790	2.4	56.7	63.0
Code	CHEAPEST	5.1745	78.9	56.7	0.0
QA	MAP	0.4184	73.4	49.1	0.0
QA	ORACLE	0.0000	100.0	49.1	0.0
QA	prune-only	0.2836	24.6	49.1	44.3
QA	quant-only	0.4905	47.8	49.1	0.0
QA	distill-only	-0.0493	27.7	49.1	63.0
QA	CHEAPEST	4.3334	64.7	49.1	0.0
All (max)	MAP	0.0288	93.8	54.7	0.0
All (max)	ORACLE	0.0000	100.0	54.7	0.0
All (max)	prune-only	0.0634	0.7	54.7	44.3
All (max)	quant-only	0.0428	93.1	54.7	0.0
All (max)	distill-only	-0.1922	4.5	54.7	63.0
All (max)	CHEAPEST	5.1782	78.2	54.7	0.0

r is nominal transformer-matrix storage relative to the dense source: pruning $r = d$ (unstructured pruning realizes this only with a sparse format; index overhead excluded); channel RTN $r = b/16$; grouped RTN $r = (b + 16/g)/16$; same-stage student $r = N_S/N_0$. Dense has $r = 1$. Distillation uses measured externally taught students, not source-specific teachers. Fixed-method source fallback can exceed the budget, making signed regret negative. No-winner share is a property of the common candidate space and repeats across policies. The multi-capability objective is $\max_c(\hat{L}_c - L_{0c})$. Five-bit OLS is underidentified in its four scored folds. Group and student coverage is incomplete; dense anchors differ by up to 0.0132 nats.

Resource-ratio models. A method-agnostic response in the storage or active-parameter ratio was tested on the heterogeneous panel: at the common storage ratio 0.5, pruning to $d = 0.5$ costs median +5.9, +8.2, and +2.9 nats while int8 quantization costs none; over 75 storage-matched pairs (14 with both sides pre-cliff) the mean absolute method gap is 9.3 nats with sign agreement 0.61, and the two shared-law hypotheses fit on pre-cliff cells reach held-out sign accuracies of 0.51 and 0.37. These models are rejected; whether another single form could unify the arms is untested. A capability- and cost-aware method selection would use predictors like these as loss constraints; an offline replay with frozen predictions quantifies the decision cost of prediction error, but no held-out selection validation was run, so nothing here is a validated selector.

H.3 Re-analysis of an earlier ability-space grid

A prior internal study fit compression laws directly on Rasch abilities from ~ 100 -item panels over a 774-cell Qwen3 grid and reported that no derived law had held-out predictive power. Our re-analysis attributes much of this to the measurement layer: (1) the $r=1$ falsification anchor was driven almost entirely by the answer-only recipe (stratified, the anchor holds and held-out error drops from $9.0\times$ to $1.8\times$ the noise floor); (2) same-benchmark evaluation inflated QA gains by roughly one third; (3) discrimination heterogeneity voids interval-scale ability while rankings survive ($\rho \geq 0.98$); (4) item-level damage is smooth pre-cliff, ruling out discrete skill deletion at the item level. These findings fixed the protocol of §2.1.

I Reproducibility and Prediction Registration

I.1 Registration and provenance codes

Each prediction test in Table 3 follows one order. The functional forms, the development set, and each standardization and coefficient are fixed and committed first. For a new source the dense capability loss is then measured, since it is a permitted pre-compression input; the predictions for each configuration and candidate are written to a versioned file and committed to the repository, with the commit hash recorded; and only then is the compressed or distilled outcome measured. Information budgets are reported separately: K0 uses pre-compression information only, K1 adds one pre-specified target-side calibration point, and post-hoc rescaling on target outcomes is reported only as an oracle diagnostic. Each comparison carries the four-part origin code defined with Table 3 and Appendix D; the label of a frozen prediction never depends on its outcome. Register files (paths in the repository; content hashes in docs/MANUSCRIPT_REWRITE_MAP.md): v36b-input-comparison/summary.json (leave-one-step-out source axis, both arms), v38-prospective/register.json (frozen step-96k coefficients), v40-prune-strength/register.json (shared power form and source-free curves), v41-distill-newpool/summary.json (the four frozen pool forms), v46-p1-newsource/predictions_frozen.json (1B@96k), v49-p1v2/register.json and predictions_*.json (1B and 6.9B pairs, protocols A and B). When a diagnostic overturns an earlier headline, the earlier claim is downgraded in the text and the prediction itself stays as committed.

I.2 Information budgets

Budget	Information available	Cost to disclose
K0, metadata only	N_0 , public D_0 , architecture, intervention configuration	metadata availability; D_0 may be undisclosed
K0 + dense anchor	the above plus the dense capability losses	probe count, one forward pass per probe, native loss units
K1	$\mathbf{L}_0$, measured once before compression the above plus one pre-specified measurement of the target at one compression configuration	the calibration configuration and its compression cost
Oracle diagnostic	post-hoc rescaling on the target outcomes	none; diagnoses form misfit, never a prediction

Table 21: Information budgets. Unknown D_0 stays unknown; handling of missing inputs is fixed on the development panel and never inferred from the target response. Predicting $\mathbf{L}_0$ from (N_0, D_0) is the task of dense scaling laws and is not attempted here.

I.3 Reproduction

All tables are generated from machine-readable result files by CPU-only scripts that run in seconds and require neither model weights nor a GPU; the repository mirrors the scripts and the result files they read, and the main prediction table is produced by a single command. Model measurements are reproducible from the recorded revisions, probe seeds, and protocols above.